

Chapter 6: Thread Scheduling

Threads are created to execute code, or at least that should be their intention. This means at some point a logical processor needs to run the thread's function. Generally, there are many threads on a typical system, but only a subset of them actually wants to execute code at the same time. Most threads are waiting for something and so are not candidates for being scheduled on processors at that time. If the number of threads that want to run (are in the *ready* state) is less or equal to the number of logical processors on the system (and there are no affinity restrictions, discussed later in this chapter), then all the ready threads simply execute.

However, some questions may come up. How long would threads get CPU time? What happens if a new thread wakes up? What about the case where there are more threads that are ready to run than there are available processors? We'll try to answer all these (and some other) questions in this chapter.

In this chapter:

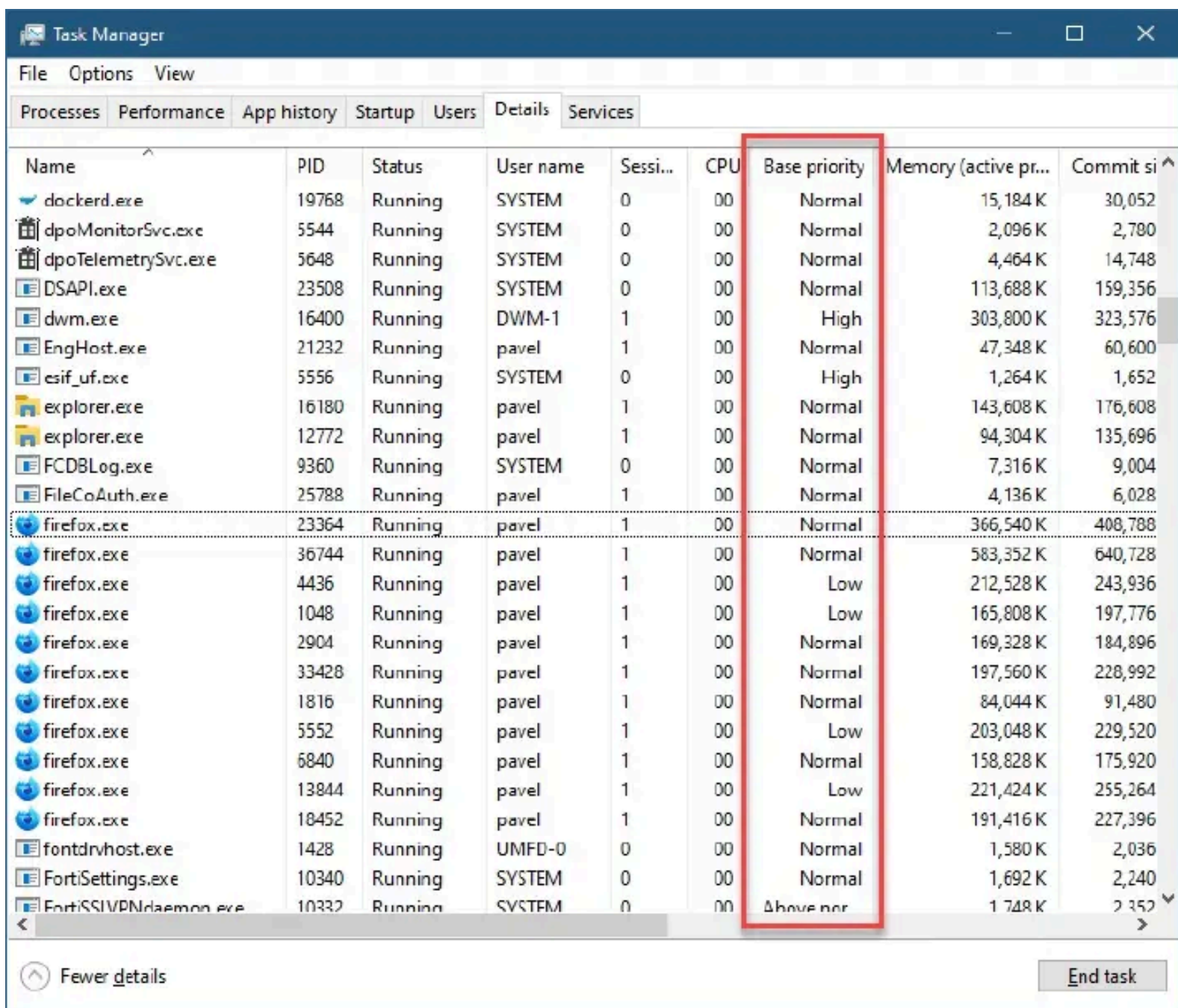
- **Priorities**
 - **Scheduling Basics**
 - **Multiprocessor Scheduling**
 - **Background Mode**
 - **Processor Groups**
 - **Suspending and Resuming**
-

Priorities

Every thread has an associated priority, that matters in the case where there are more threads that want to execute than there are available processors. In this section we'll look at the

available priorities and how these can be manipulated, and the next section we'll see how these apply in scheduling.

Thread priorities are from 0 to 31, where 31 is the highest. Technically, thread 0 is reserved for a special thread called *zero page thread* that is part of the memory manager in the kernel. It's the only thread that is allowed to have a priority of zero. So technically, usable priorities are from 1 to 31. In user mode (where we write code in this book), the priorities cannot be set to an arbitrary value. Instead, a thread's priority is a combination of a process' *Priority Class* (called *Base Priority* in *Task Manager*) and an offset around that base priority. Figure 6-1 shows *Task Manager* with the *Base Priority* column highlighted.



Name	PID	Status	User name	Sessi...	CPU	Base priority	Memory (active pr...	Commit si ^
dockerd.exe	19768	Running	SYSTEM	0	00	Normal	15,184 K	30,052
dpoMonitorSvc.exe	5544	Running	SYSTEM	0	00	Normal	2,096 K	2,780
dpoTelemetrySvc.exe	5648	Running	SYSTEM	0	00	Normal	4,464 K	14,748
DSAPI.exe	23508	Running	SYSTEM	0	00	Normal	113,688 K	159,356
dwm.exe	16400	Running	DWM-1	1	00	High	303,800 K	323,576
EngHost.exe	21232	Running	pavel	1	00	Normal	47,348 K	60,600
esif_uf.exe	5556	Running	SYSTEM	0	00	High	1,264 K	1,652
explorer.exe	16180	Running	pavel	1	00	Normal	143,608 K	176,608
explorer.exe	12772	Running	pavel	1	00	Normal	94,304 K	135,696
FCDBLog.exe	9360	Running	SYSTEM	0	00	Normal	7,316 K	9,004
FileCoAuth.exe	25788	Running	pavel	1	00	Normal	4,136 K	6,028
firefox.exe	23364	Running	pavel	1	00	Normal	366,540 K	408,788
firefox.exe	36744	Running	pavel	1	00	Normal	583,352 K	640,728
firefox.exe	4436	Running	pavel	1	00	Low	212,528 K	243,936
firefox.exe	1048	Running	pavel	1	00	Low	165,808 K	197,776
firefox.exe	2904	Running	pavel	1	00	Normal	169,328 K	184,896
firefox.exe	33428	Running	pavel	1	00	Normal	197,560 K	228,992
firefox.exe	1816	Running	pavel	1	00	Normal	84,044 K	91,480
firefox.exe	5552	Running	pavel	1	00	Low	203,048 K	229,520
firefox.exe	6840	Running	pavel	1	00	Normal	158,828 K	175,920
firefox.exe	13844	Running	pavel	1	00	Low	221,424 K	255,264
firefox.exe	18452	Running	pavel	1	00	Normal	191,416 K	227,396
fontdrvhost.exe	1428	Running	UMFD-0	0	00	Normal	1,580 K	2,036
FortiSettings.exe	10340	Running	SYSTEM	0	00	Normal	1,692 K	2,240
FortiSSVPNdaemon.exe	10332	Running	SYSTEM	0	00	Above nor	1,748 K	2,352

Figure 6-1: Base Priority in *Task Manager*

Each priority class is associated with a priority value, shown in table 6-1.

Table 6-1: Process priority classes

Priority Class	Priority value	API constant
Idle (Low)	4	IDLE_PRIORITY_CLASS
Below Normal	6	BELOW_NORMAL_PRIORITY_CLASS
Normal	8	NORMAL_PRIORITY_CLASS
Above Normal	10	ABOVE_NORMAL_PRIORITY_CLASS
High	13	HIGH_PRIORITY_CLASS
Real-time	24	REALTIME_PRIORITY_CLASS

The name “real-time” in table 6-1 does not imply Windows is a real-time operating system; it’s not. Windows cannot provide latency and timing guarantees that real-time operating systems do. This is because Windows works with a wide variety of hardware and so it’s simply not possible to have any such guarantees when hardware range is unconstrained. “Real-time” in table 6-1 just means “higher than all the rest”.

The priority class of a process can be set when that process is being created with `CreateProcess`. The sixth parameter (flags) can be combined with one of the constants in table 6-1. If no explicit priority class is specified, the priority class defaults to *Normal*, unless the creator’s priority class is *Idle* or *Below Normal*, in which case the creator’s priority class is used.

If a process already exists, changing the priority class is possible with `SetPriorityClass`:

```
BOOL SetPriorityClass(
    _In_ HANDLE hProcess,
    _In_ DWORD dwPriorityClass);
```

Task Manager as well as *Process Explorer* offer a context menu to change a process’ priority class.

The process handle in question must have the `PROCESS_SET_INFORMATION` access mask for the call to succeed. Also, if the target priority class is Real-time, the caller must have the `SeIncreaseBasePriority` privilege. If it does not, the function does not fail, but the resulting priority class is set to *High* rather than *Real-time*.

Naturally, the opposite function exists as well to retrieve the priority class of a process:

```
DWORD GetPriorityClass(_In_ HANDLE hProcess);
```

The handle access mask needs only `PROCESS_QUERY_LIMITED_INFORMATION`, which can be obtained for almost all processes.

The process priority class has no direct effect on the process itself, since a process does not run - threads do. All threads created in a process have their default priority set to the priority class level. For example, in a *Normal* priority class process, all threads have a default priority of 8. To make changes to a thread's priority, the `SetThreadPriority` function can be used:

```
BOOL SetThreadPriority(
    _In_ HANDLE hThread,
    _In_ int     nPriority);
```

The *nPriority* parameter is not an absolute priority value. Instead, it's one of seven possible values (except for *Real-time* priority class, see next sidebar), depicted in table 6-2.

Table 6-2: Thread relative priorities

Priority value	Effect
<code>THREAD_PRIORITY_IDLE</code> (-15)	priority drops to 1 except for real-time priority class, where thread priority drops to 16
<code>THREAD_PRIORITY_LOWSET</code> (-2)	priority drops by 2 relative to the priority class
<code>THREAD_PRIORITY_BELOW_NORMAL</code> (-1)	priority drops by 1 relative to the priority class
<code>THREAD_PRIORITY_NORMAL</code> (0)	priority set to the process priority class value
<code>THREAD_PRIORITY_ABOVE_NORMAL</code> (1)	priority increases by 1 relative to the priority class
<code>THREAD_PRIORITY_HIGHEST</code> (2)	priority increases by 2 relative to the priority class
<code>THREAD_PRIORITY_TIME_CRITICAL</code> (15)	priority increases to 15 except for the real-time priority class, where the thread priority increases to 31



The real-time priority class is special in relation to table 6-2. Threads in that priority class can be assigned any value from 16 to 31. `SetThreadPriority` accepts the values from -7 to -3 and from 3 to 7, corresponding to priorities 17 to 21 and 27 to 30.

The *High* priority class has only six levels. That's because threads in processes with priority class other than real-time cannot have their priorities above 15.

The *idle* and *time critical* values are called *Saturation* values.

The net effect of table 6-1 and table 6-2 can be summarized in table 6-3 and figure 6-2. In figure 6-2, each rectangle signifies a possible thread priority value based on SetPriorityClass/SetThreadPriority. Table 6-3 is a summary table representation of tables 6-1 and 6-2 combined.

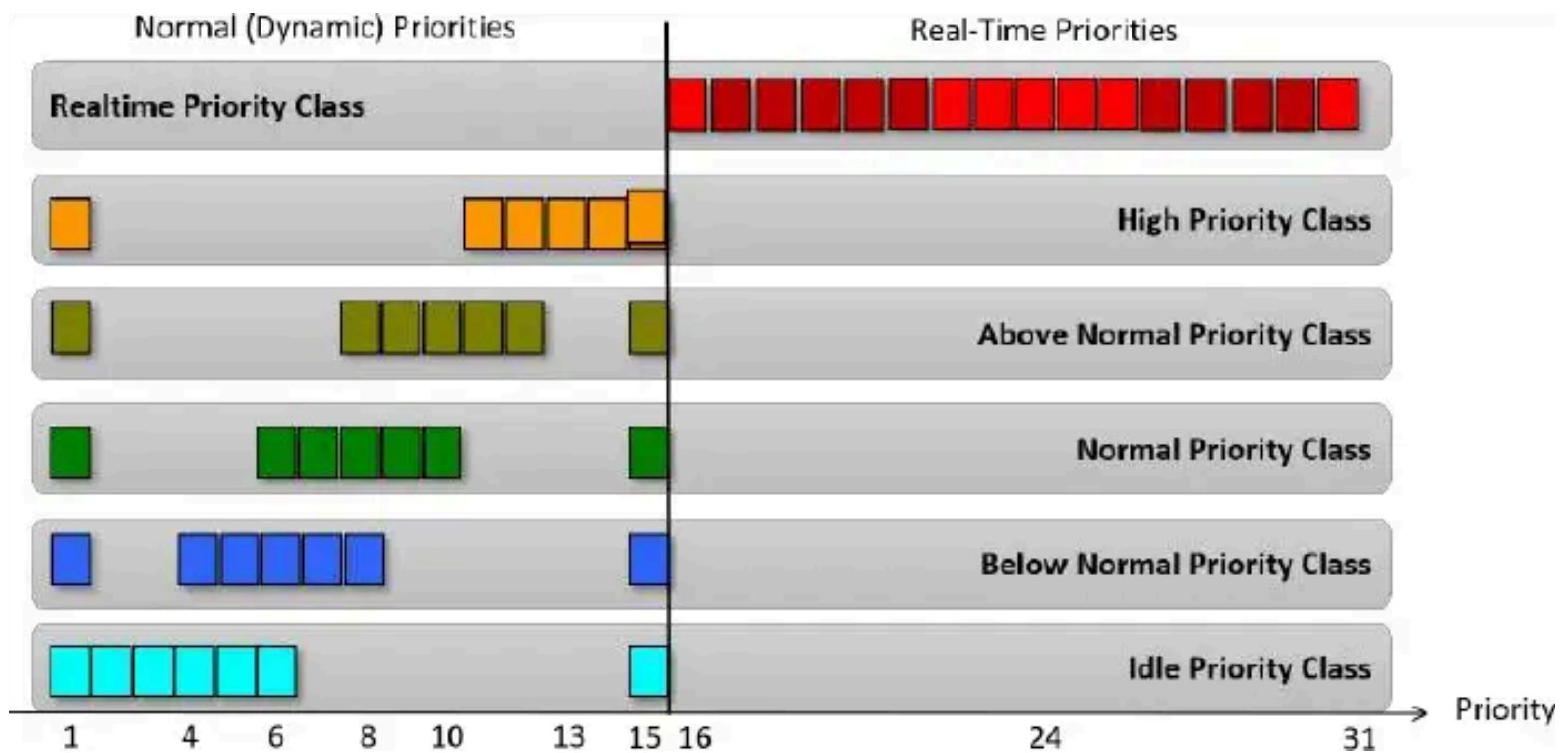


Figure 6-2: Thread Priorities

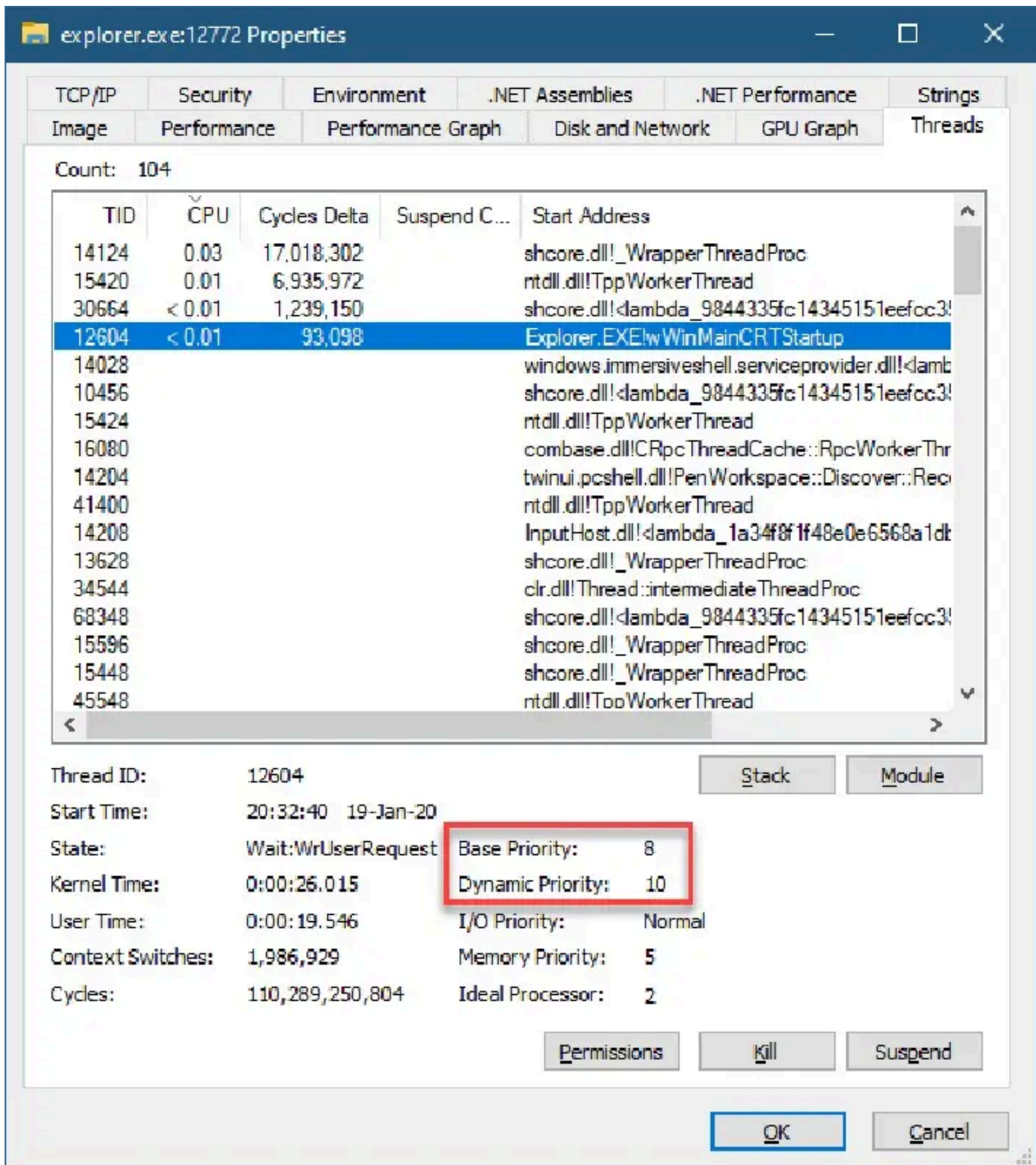
Table 6-3: Thread priorities by priority class

Priority Class (right) Relative Priority (down)	Real- Time	High	Above Normal	Normal	Below Normal	Idle
Time Critical (+15)	31	15	15	15	15	15
Highest (+2)	26	15	12	10	8	6
Above Normal (+1)	25	14	11	9	7	5
Normal (0)	24	13	10	8	6	4
Below Normal (-1)	23	12	9	7	5	3
Lowest (-2)	22	11	8	6	4	2
Idle (-15)	16	1	1	1	1	1

The resulting combination of process priority class and relative thread priority is the final thread's priority. From the kernel's scheduler perspective, only the final number is important. It doesn't care how that number came to be. For example, priority 8 can be reached in one of three ways: Normal priority class with normal (0) relative thread priority; Below normal priority class with highest (+2) relative thread priority; Above normal priority class with lowest (-2) relative thread priority. From the scheduler's perspective, they are all the same; it doesn't care about processes in general, only threads.

The real-time priority range (16-31) is used by many kernel threads that do essential work for the system as a whole, so it's important for processes whose threads run in that range not to consume too much CPU time. Of course, a process must have a very good reason to be in that range in the first place.

If we look at thread priorities in a tool such as *Process Explorer*, we find two values for the priority: *Base Priority* and *Dynamic priority* (figure 6-3).

Figure 6-3: Thread properties in *Process Explorer*

The base priority is the one set by the developer (or the default value), while the dynamic priority is the actual current priority for that thread. In some cases the priority is increased (boosted) temporarily. We'll look at some of the reasons for this boost later in this chapter.

From the scheduler's perspective, the dynamic priority is the determining priority value.



Threads in the real-time range never have their priority boosted.

Scheduling Basics

Scheduling in general is quite complex, taking into account several factors, some of them conflicting: multiple processors, power management (the desire to save power on the one hand and utilize all processors on the other hand), NUMA (Non-Uniform Memory Architecture), hyper-threading, caching, and more. The exact scheduling algorithms are undocumented for a reason: Microsoft can make modifications and tweaks in subsequent Windows versions and updates without developers taking any dependency on the exact algorithms. Having said that, it's possible to experience many of the scheduling algorithms by experimentation.

We'll start with the simplest scheduling possible - when there is a single processor on a system, since it's fundamental to how scheduling works. Later, we'll look at some of the ways these algorithms change on a multi-processing system.

Single CPU Scheduling

The scheduler maintains a *Ready Queue*, where threads that want to execute (are in the *Ready* state) are managed. All other threads that do not want to execute at this time (in the *Wait* state) are not being looked at since they don't want to execute. Figure 6-4 shows an example system where seven threads are in the *ready* state. They are arranged in a number of queues based on their priority in.

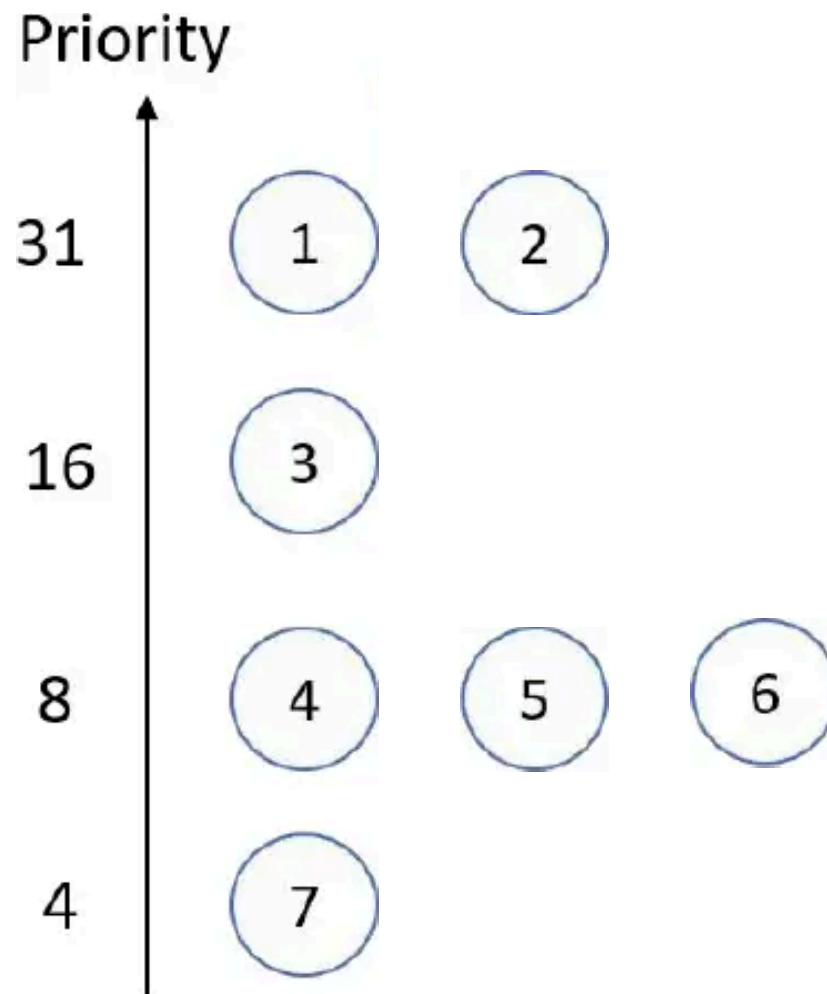


Figure 6-4: Threads in *Ready* state

There may be thousands of threads on a system, but most are in the waiting state, so are not considered by the scheduler.

The algorithm for a single CPU goes like this:

1. The highest priority thread runs first. In figure 6-4, threads 1 and 2 have the highest (and same) priority (31), so the first thread in the queue for priority 31 runs; let's assume it's thread 1 (figure 6-5).

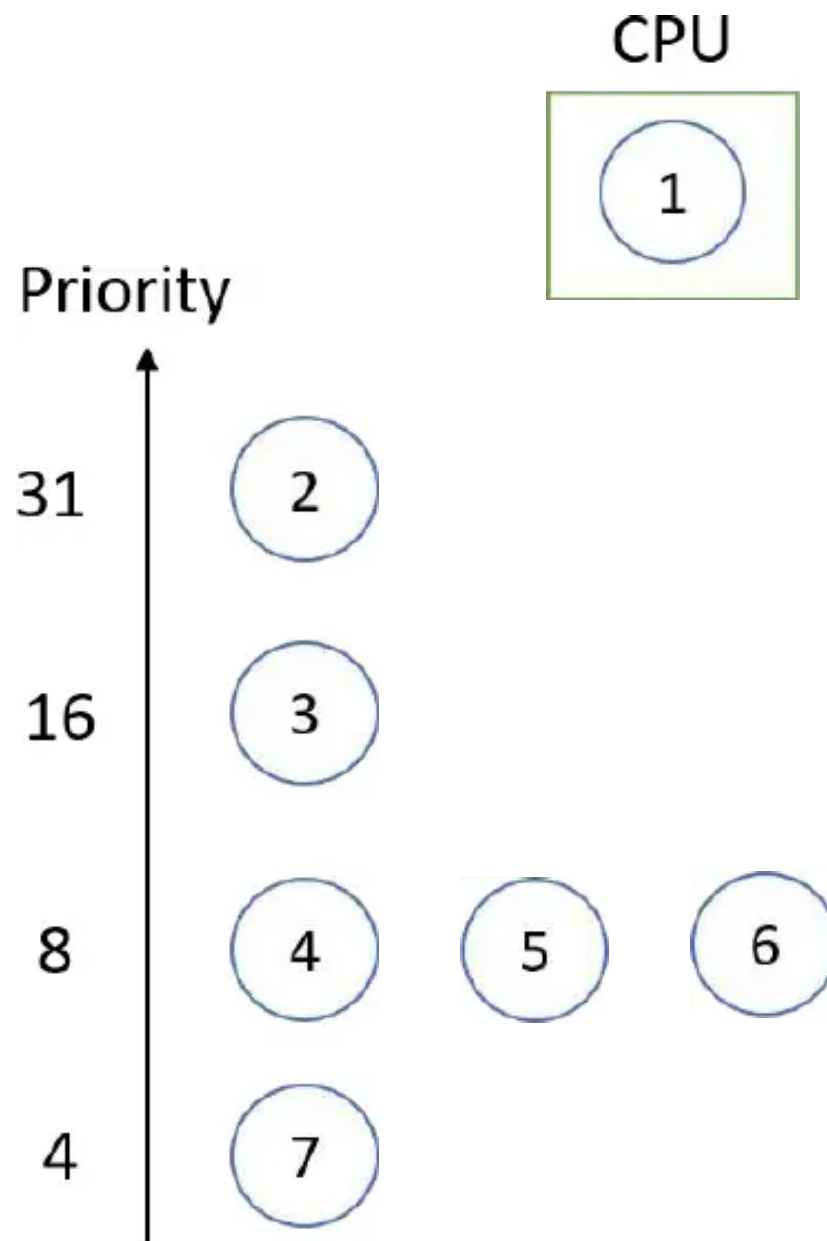


Figure 6-5: Highest priority thread runs

Thread 1 runs for a certain amount of time called a *Quantum*. The length of a quantum is discussed in the next section. Assuming thread 1 has lots to do, when its quantum expires, the scheduler *preempts* thread 1, saving its state in its kernel stack, and it goes back to the *Ready* state (since it still has things to do). Thread 2 now becomes the running thread since it has the same priority (figure 6-6).

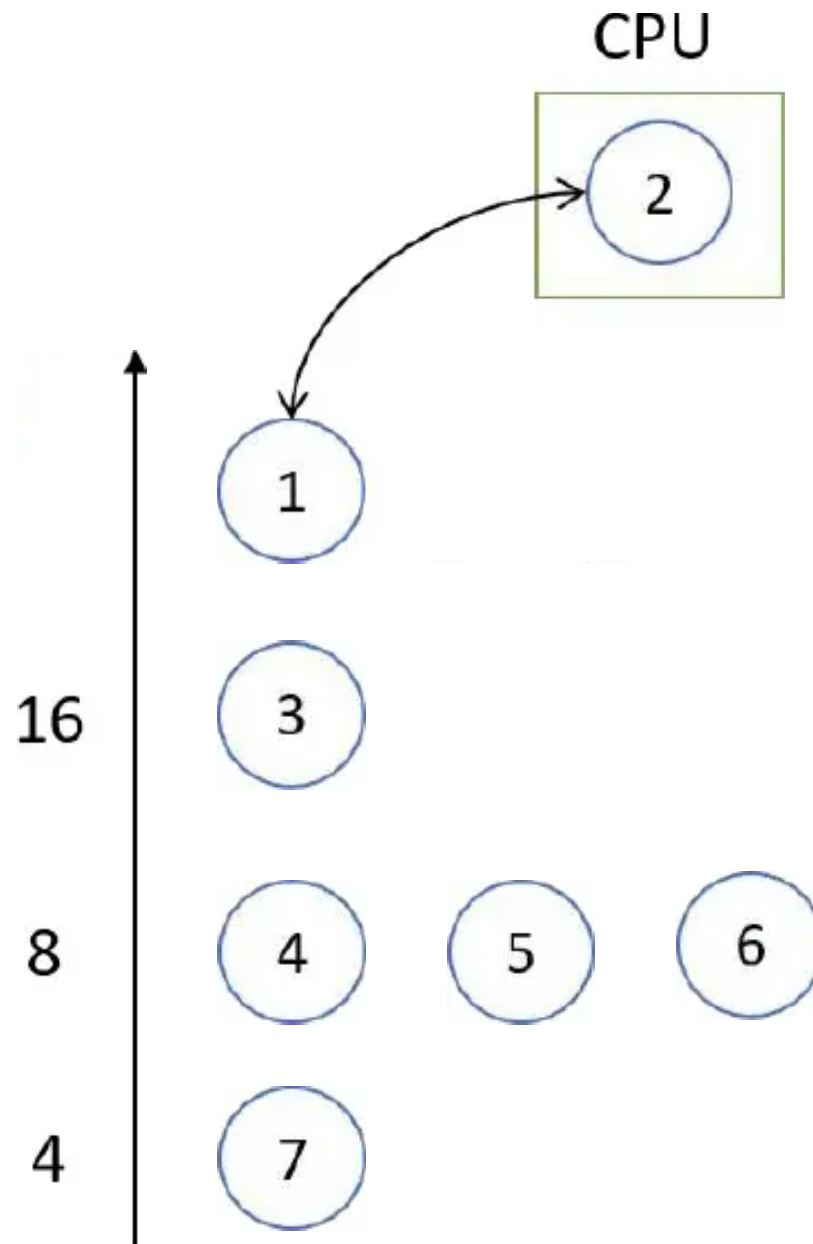


Figure 6-6: Thread 2 is running, thread 1 is back to ready

So, the priority is the determining factor. As long as threads 1 and 2 need to execute, they will round-robin on the CPU, each running for a *quantum*. Fortunately, threads typically don't run forever. Instead, they enter a wait state at some point. Here are a few examples that cause a thread to enter a wait state:

- Performing a synchronous I/O operation
- Waiting on a kernel object, that is currently not signaled
- Waiting for a UI message when there are none
- Entering a voluntary sleep

Once a thread enters *wait* state, it's removed from the scheduler's ready queue. Let's assume that threads 1 and 2 entered a wait state. Now the highest priority thread is thread 3 and it becomes the running thread (figure 6-7).

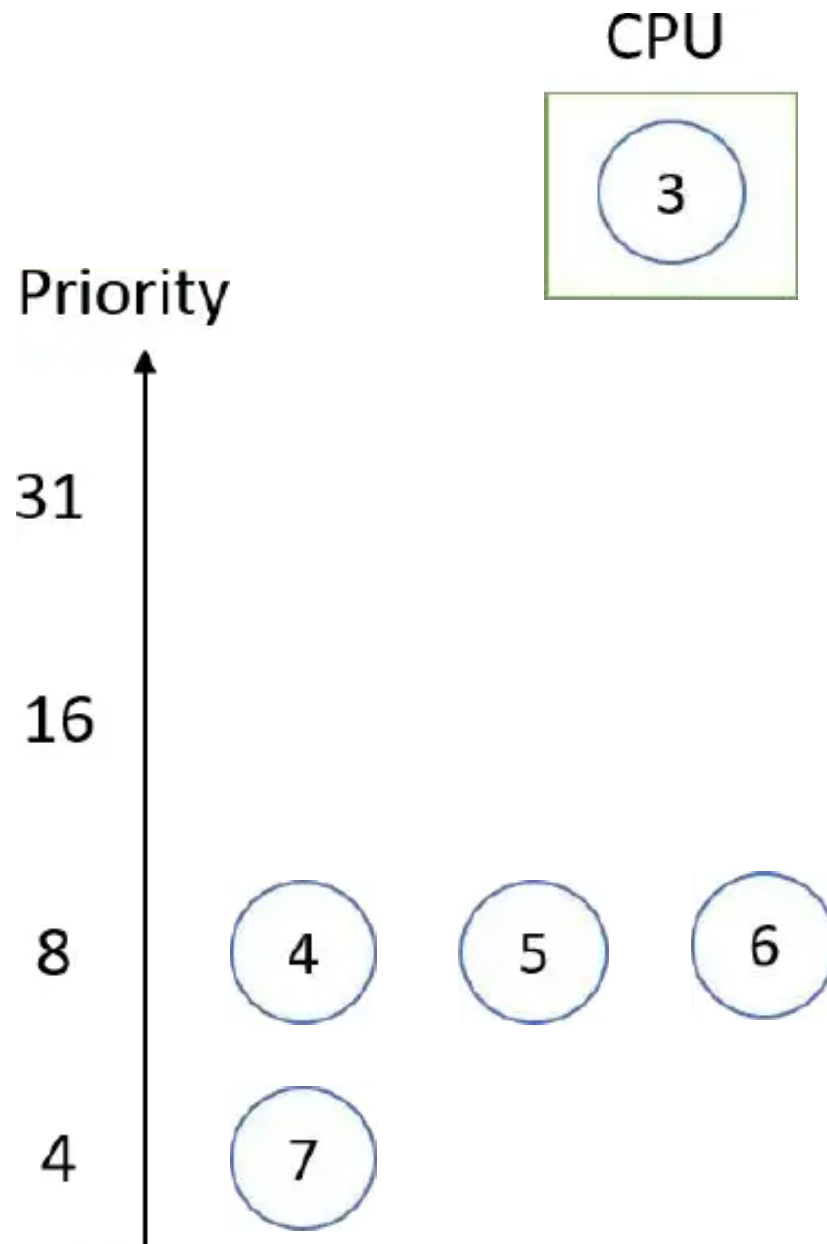


Figure 6-7: Thread 3 is running

Thread 3 runs for a quantum. If it still has work to do, it gets another quantum, since it's the only one in its priority level. If, however, thread 1 receives whatever it was waiting for, it goes to the ready state and preempts thread 3 (since thread 1 has a higher priority) and becomes the running thread. Thread 3 goes back to the ready state (figure 6-8). This switch is not at the end of thread 3's quantum, but rather at the time of the change (thread 1 finishing waiting). Thread 3's quantum is replenished if its priority is above 15 (which it is in this example).



If the preempted thread's priority is 16 or higher, it gets its quantum restored when it goes back to the ready state.

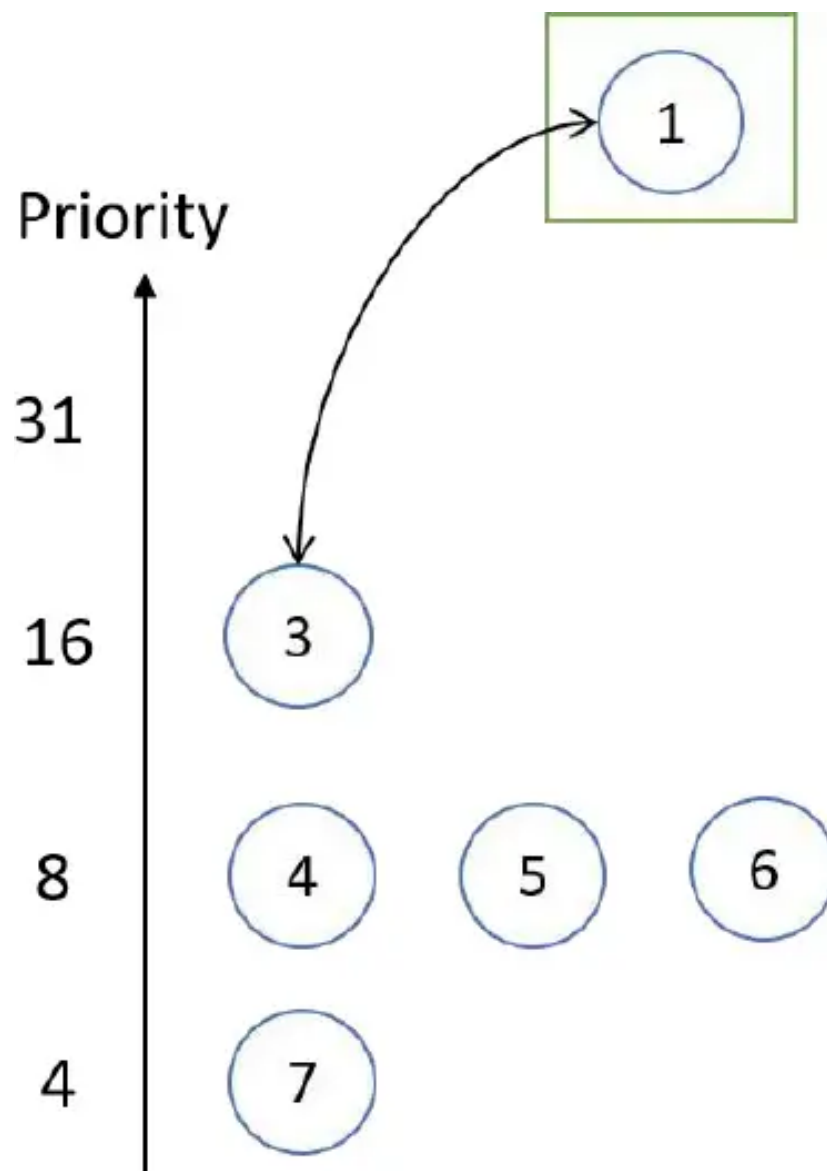


Figure 6-8: Thread 1 is running, thread 3 is back to ready

With this algorithm in mind, threads 4, 5 and 6 will run each having their own quantum if there are no higher-priority threads in the ready state.

This is the basis of scheduling. In fact, in a real single CPU scenario, this is exactly the algorithm used. However, even in this case, Windows tries to be “fair” to some extent. For example, thread 7 in figures 6-4 to 6-8 (with a priority of 4) may not run if higher-priority threads are in the ready state, and so it suffers from *CPU starvation*. Is that thread doomed in such a system? Not really; the system will boost the thread’s priority to 15 every roughly 4 seconds, giving it a better chance to make forward progress. This boost lasts for one quantum of actual execution of the thread, and then the priority drops back to its initial value. This is just one example of a temporary priority boost. You’ll see other examples in the section “Priority Boosts” later in this chapter.

The Quantum

The quantum was mentioned a few times in the preceding section - but how long is a quantum? The scheduler works in two orthogonal ways: the first is with a timer, that fires every 15.625 milliseconds by default, and can be obtained by calling `GetSystemTimeAdjustment` and looking at the second argument. Another way is to use the *clockres* tool from *SysInternals*:

```
C:\Users\pavel>clockres
```

```
Clockres v2.1 - Clock resolution display utility  
Copyright (C) 2016 Mark Russinovich  
Sysinternals
```

```
Maximum timer interval: 15.625 ms  
Minimum timer interval: 0.500 ms  
Current timer interval: 1.000 ms
```

The value to look at in relation to the quantum is the *Maximum time interval* value.

The *Current time interval* from *clockres* shows the current timer firing interval. This is typically lower than the maximum interval because of multimedia timers that may have been requested. This allows getting timer notifications up to a 1 msec resolution. Regardless, the quantum itself is not affected by the Current timer interval.

The default quantum is 2 clock ticks for client machines (Home, Pro, Enterprise, XBOX, etc.) and 12 clock ticks for server machines. In other words, the quantum is 31.25 msec for client and 187.5 msec for server.

The reason server versions get a longer quantum is to increase the chances of a client request being handled completely in a single quantum. This is less of a concern on a client machine since it may have many processes doing relatively little work each, some with user interface that should be responsive, so short quanta better fit the bill. Switching from client to server (or vice versa) in terms of quantum can be achieved with the following dialog which I like to call “the most incomprehensible dialog in Windows” (figure 6-9).

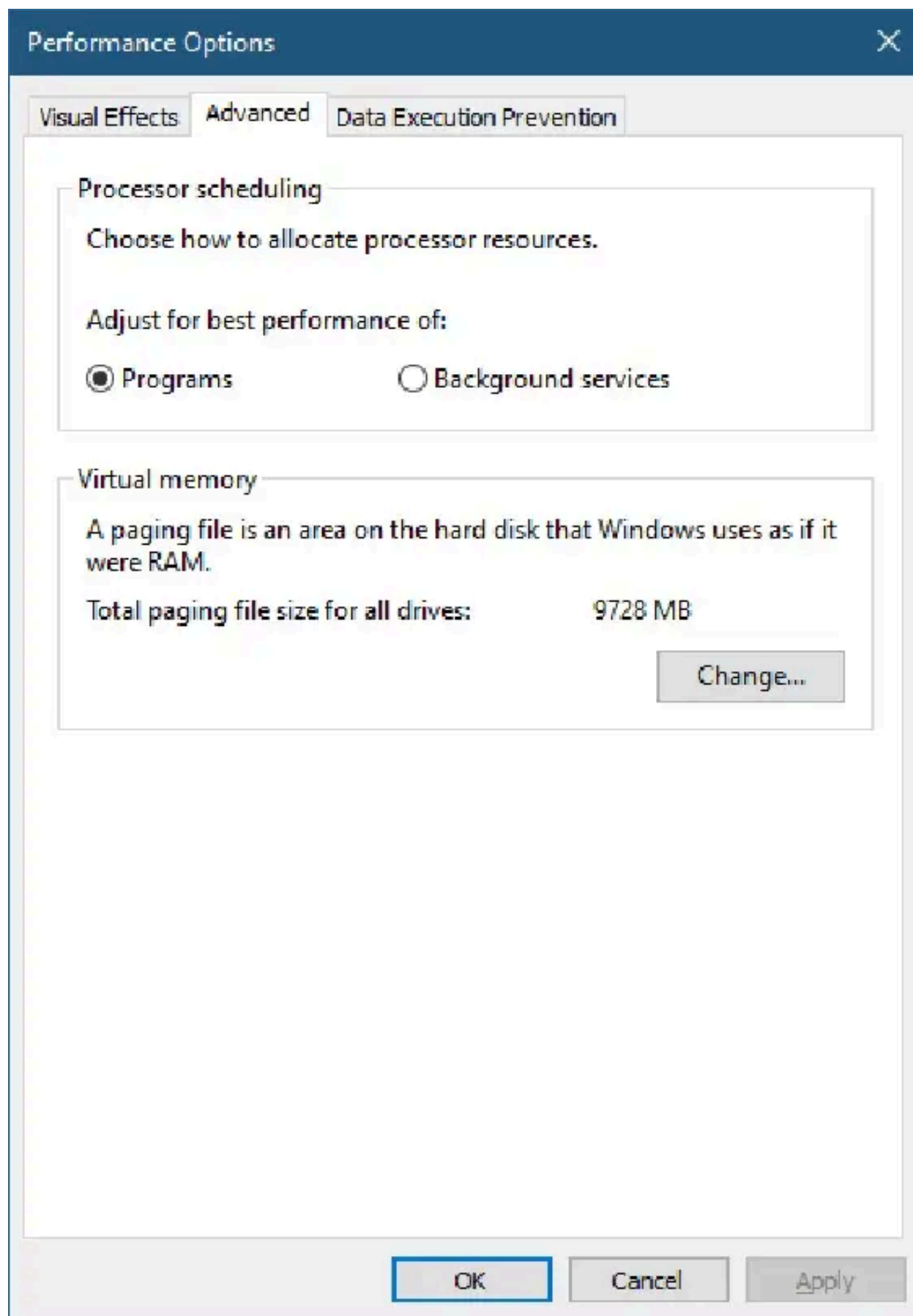


Figure 6-9: Performance Options dialog



Here is how to get to this dialog: go to System properties (from the Control Panel or right-click *This PC* in *Explorer* and select *Properties*). Then click *Advanced System Settings*. A dialog opens - click the *Performance Settings* button. The Performance Options dialog appears. Click the *Advanced* tab, and you're there.

This dialog has two parts which are completely unrelated. The lower part controls the size of page files (if any), discussed later in this book. The upper part is the “incomprehensible” part. The option *Programs* means short quanta, while the option *Background Services* means long quanta. If you check the other radio button, the change is instantaneous.



There is yet another difference between the two options. *Programs* also means that in the foreground process (the one hosting the foreground window), all threads get triple quantum by default. This quantum stretching effect is not present for *Background Services* because a true server is unlikely to have an interactive user sitting at the console. Also, some of the priority boosts we'll look at later in this chapter do not apply when *Background Services* is selected.

There are other ways to change the quantum. For fine-grained control, the registry value *HKLM\SYSTEM\CurrentControlSet\Control\PriorityControl\Win32PrioritySeparation* controls not just the length of a quantum, but also quantum stretching for the foreground process. See the “Windows Internals” book (chapter 4) for the details. It's best to leave this value at its default, where the rules previously described apply.

Another way to control the quantum is available by using a job object (described in detail in chapter 4) with the *SchedulingClass* field in *JOB_OBJECT_BASIC_LIMIT_INFORMATION*. This works for long fixed quantum systems only (the default for server systems). The scheduling class value (between 0 and 9) sets the quantum for threads in processes that are part of the job in question in the following way:

$$\text{Quantum} = 2 * (\text{timer interval}) * (\text{Scheduling class} + 1);$$

The default scheduling class is 5, effectively giving a quantum of $12 * \text{timer interval}$, which is the default on server systems, as we saw earlier. The highest value (9) causes the threads to be non-preemptive, meaning they have an infinite quantum (can continue running theoretically indefinitely until they voluntarily enter a wait state). A value higher than 5 requires the caller to have the *SeIncreaseBasePriority* privilege, which by default is available for users in the Administrators group, but not to standard users.



The scheduling class value only applies for processes with priority classes higher than *Idle*.

Processor Groups

The original Windows NT design supported 32 processors at most, with a machine word (32 bit) used to indicate the actual processors on a system, with each bit representing a processor. When 64 bit Windows appeared, the maximum number of processors was naturally extended to 64.

Starting with Windows 7 (64 bit systems only), Microsoft wanted to support more than 64 processors, and so an additional parameter entered the scene: a *processor group*. For example, Windows 7 and Server 2008 R2 support up to 256 processors, meaning there are 4 processor groups on a system with 256 processors.



Windows 8 and Server 2012 support 640 processors (10 groups) and Windows 10 supports even more. The basic rule remains - at most 64 processors per group - the number of groups increases as needed.

A thread can be a member of one processor group, which means a thread can be scheduled on one of (at most) 64 processors that are part of its current group. When a process is created, it's assigned a processor group in a round-robin fashion, so that processes are "load balanced" across groups. Threads in a process are assigned to the process group. A parent process can affect the initial processor group of a child process in one of the following ways:

1. The parent process can use the `INHERIT_PARENT_AFFINITY` flag as one of the flags to `CreateProcess` to indicate the child process should inherit its parent processor group rather than getting it according to the round robin managed by the system. If the parent process' threads use more than one affinity group, one of the groups is selected arbitrarily as the one used for the child process group.
2. The parent process can use the `PROC_THREAD_ATTRIBUTE_GROUP_AFFINITY` process attribute to specify a desired default processor group.

The process group affinity can be retrieved with `GetProcessGroupAffinity`:

```
BOOL GetProcessGroupAffinity(  
    _In_     HANDLE  hProcess,  
    _Inout_  PUSHORT GroupCount,  
    _Out_    PUSHORT GroupArray);
```

Controlling the processor group of a specific thread is possible with the `SetThreadGroupAffinity` function. This is discussed in the section “Hard Affinity” later in this chapter.

Multiprocessor Scheduling

Multiprocessor scheduling adds complexity to the scheduling algorithms. The only thing Windows guarantees is that the highest priority thread that wants to execute (at least one if there’s more than one) is currently running. In this section, we’ll take a look at some of the parameters that affect scheduling.

Affinity

Normally, a thread may be scheduled on any processor. However, the *affinity* of a thread, that is, the processors it’s allowed to run on can be controlled in several ways, described in the followings sections.

Ideal Processor

The *Ideal Processor* is an attribute of a thread, also sometimes referred to as “Soft Affinity”. An ideal processor serves as a hint to the scheduler as - all other things being equal - the preferred processor for executing code for that thread. The default ideal processor is selected in a round-robin fashion, starting with a random generated when the process is created. On a hyper-threaded system, the next ideal processor is selected from the next core, rather than from the next logical processor.

The ideal processor can be viewed with the *Process Explorer* tool, as one of the properties shown in the *Threads* tab (figure 6-10).

Thread ID:	4764		
Start Time:	19:41:30	29-Jan-20	
State:	Wait:UserRequest	Base Priority:	8
Kernel Time:	0:00:00.000	Dynamic Priority:	8
User Time:	0:00:00.093	I/O Priority:	Normal
Context Switches:	1,416	Memory Priority:	5
Cycles:	81,424,056	Ideal Processor:	2

Figure 6-10: Thread's Ideal processor in *Process Explorer*

A thread's ideal processor can be changed with `SetThreadIdealProcessor`:

```
DWORD WINAPI SetThreadIdealProcessor(
    _In_ HANDLE hThread,
    _In_ DWORD dwIdealProcessor);
```

The ideal processor changed by this function is between 0 and the maximum number of processors minus 1, where 63 is the maximum, since this is the highest processor number in any group. If more than one group is supported on the system, the current thread's group is used. The function returns the previous ideal processor number or -1 (0xffffffff) in case of an error. Passing the special value `MAXIMUM_PROCESSORS` (which equals 32 on 32 bit systems and 64 on 64 bit systems) for the ideal processor just returns the current ideal processor.

`SetThreadIdealProcessor` changes the ideal processor for the current processor group the thread is part of. To make the change for a different group, the extended `SetThreadIdealProcessorEx` function can be used:

```
typedef struct _PROCESSOR_NUMBER {
    WORD Group;
    BYTE Number;
    BYTE Reserved;
} PROCESSOR_NUMBER, *PPROCESSOR_NUMBER;
```

```
BOOL SetThreadIdealProcessorEx(
    _In_ HANDLE hThread,
    _In_ PPROCESSOR_NUMBER lpIdealProcessor,
    _Out_opt_ PPROCESSOR_NUMBER lpPreviousIdealProcessor);
```

The `PROCESSOR_NUMBER` structure's `Group` member is the group to set the ideal processor in, and the `Number` member is the CPU index (0 to 63). As with the non-Ex function, the previous ideal processor can be retrieved using the last optional parameter.

Hard Affinity

While the ideal processor serves as a hint and a recommendation on which processor a thread should execute, *hard affinity* (sometimes called just *affinity*) allows specifying the allowed processors to execute on for a particular thread or process. Hard affinity works on two levels: process and thread, where the basic rule is that a thread cannot “escape” the affinity set by its process.



Generally speaking, setting hard affinity constraints is usually a bad idea. It limits the scheduler's freedom in assigning processors and can cause threads to get less CPU time than if they had no hard affinity constraints. Still, in some rare occasions this can be useful, as threads that run on the same set of processors are more likely to get better CPU cache utilization. This could be useful for systems that run specific known processes, rather than some random machine that may be running anything. Another use of hard affinity is in stress testing, such as using fewer processors for some execution to see how a system with that restricted number of processors might behave when running the same process(es).

Setting a process-wide hard affinity is achieved with `SetProcessAffinityMask`:

```
BOOL WINAPI SetProcessAffinityMask(  
    _In_ HANDLE    hProcess,  
    _In_ DWORD_PTR dwProcessAffinityMask);
```

The process handle must have the `PROCESS_SET_INFORMATION` access mask. The affinity mask itself is a bit mask where a bit set to one indicates allowed processor and a zero bit indicates a forbidden processor. For example, the affinity mask `0x1a` (11010 in binary) indicates processors 1, 3 and 4 are the only ones allowed. The function changes the affinity mask for the current process processor group.

Task Manager and *Process Explorer* allow changing a process affinity mask. For *Task Manager*, right clicking a process in the *Details* tab and selecting *Set affinity* shows a dialog with the available processors for the current process processor group (system affinity mask) (figure 6-11). Clicking *OK* calls `SetProcessAffinityMask` to set the new affinity mask. *Process Explorer* has similar functionality.

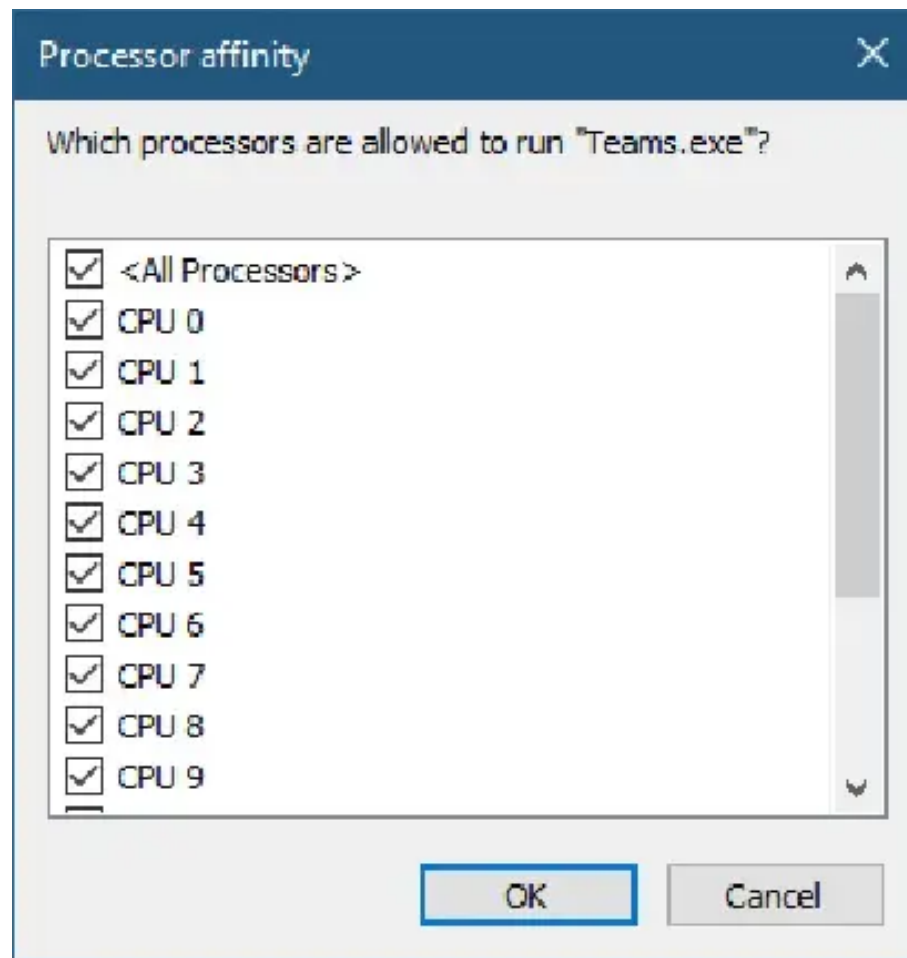


Figure 6-11: Setting hard affinity in *Task Manager*

Naturally, the inverse function is available as well, which also provides the system affinity mask:

```
BOOL WINAPI GetProcessAffinityMask(
    _In_ HANDLE hProcess,
    _Out_ PDWORD_PTR lpProcessAffinityMask,
    _Out_ PDWORD_PTR lpSystemAffinityMask);
```

Here is an example for getting the current process affinity mask, perhaps just to retrieve the system affinity mask:

```
DWORD_PTR processAffinity, systemAffinity;
::GetProcessAffinityMask(::GetCurrentProcess(), &processAffinity,
    &systemAffinity);
```

As an example, on a system with 16 logical processors, the returned system affinity mask is 0xffff.

Setting the process affinity mask constraints all the threads in the process to use that mask. An individual thread can further restrict its affinity mask by calling `SetThreadAffinityMask`:

```
DWORD_PTR WINAPI SetThreadAffinityMask(
    _In_ HANDLE hThread,
    _In_ DWORD_PTR dwThreadAffinityMask);
```

The function sets the thread affinity mask, if possible. Remember the basic rule: a thread's affinity mask cannot include processors not specified in its process affinity mask. The return value is the previous affinity mask of the thread, or zero if an error occurs.

On a system with more than 64 processors, a thread may change its processor group while specifying an affinity mask with `SetThreadGroupAffinity`:

```
typedef struct _GROUP_AFFINITY {
    KAFFINITY Mask;           // affinity bit mask
    WORD Group;               // group number
    WORD Reserved[3];
} GROUP_AFFINITY, *PGROUP_AFFINITY;

BOOL SetThreadGroupAffinity(
    HANDLE hThread,
    const GROUP_AFFINITY *GroupAffinity,
    PGROUP_AFFINITY PreviousGroupAffinity);
```

The function can do two things: change the processor group for the specified thread and/or change the hard affinity mask in that group. If the group is changed, it becomes the default processor group for the thread's process. This complicates things, so it's usually better to make sure all threads in a process are part of the same group. Still, if more than 64 threads may be running concurrently in that process (and there are more than 64 processors on the system), then changing the processor group of some threads can be beneficial, as they can utilize processors from another group.



KAFFINITY is typedefed as ULONG_PTR.

As you might expect, the inverse function is available as well:

```

BOOL GetThreadGroupAffinity(
    HANDLE          hThread,
    PGROUP_AFFINITY GroupAffinity);

```

CPU Sets

As we’ve seen in the previous section, a thread’s affinity cannot “escape” its process affinity. However, there are some scenarios where it’s beneficial to have a thread (or threads) use processors that other threads in the process are forbidden to use. Windows 10 and Server 2016 added this capability, known as *CPU Sets*.

The term “CPU Set” indicates an abstract view of processors, where each CPU set is potentially mapped to one or more logical processors. Currently, however, each CPU set maps exactly to a single logical processors. The system has its own CPU sets, which by default includes all processors on a system. This information is available with `GetSystemCpuSetInformation`:

```

BOOL WINAPI GetSystemCpuSetInformation(
    _Out_opt_ PSYSTEM_CPU_SET_INFORMATION Information,
    _In_      ULONG                      BufferLength,
    _Out_     PULONG                     ReturnedLength,
    _In_opt_  HANDLE                     Process,
    _Reserved_ ULONG                     Flags);

```

The function returns an array of structures of type `SYSTEM_CPU_SET_INFORMATION` defined like so:

```

typedef struct _SYSTEM_CPU_SET_INFORMATION {
    DWORD          Size;
    CPU_SET_INFORMATION_TYPE Type;    // currently
    union {
        struct {
            DWORD   Id;
            WORD    Group;
            BYTE    LogicalProcessorIndex;
            BYTE    CoreIndex;
            BYTE    LastLevelCacheIndex;
            BYTE    NumaNodeIndex;
            BYTE    EfficiencyClass;

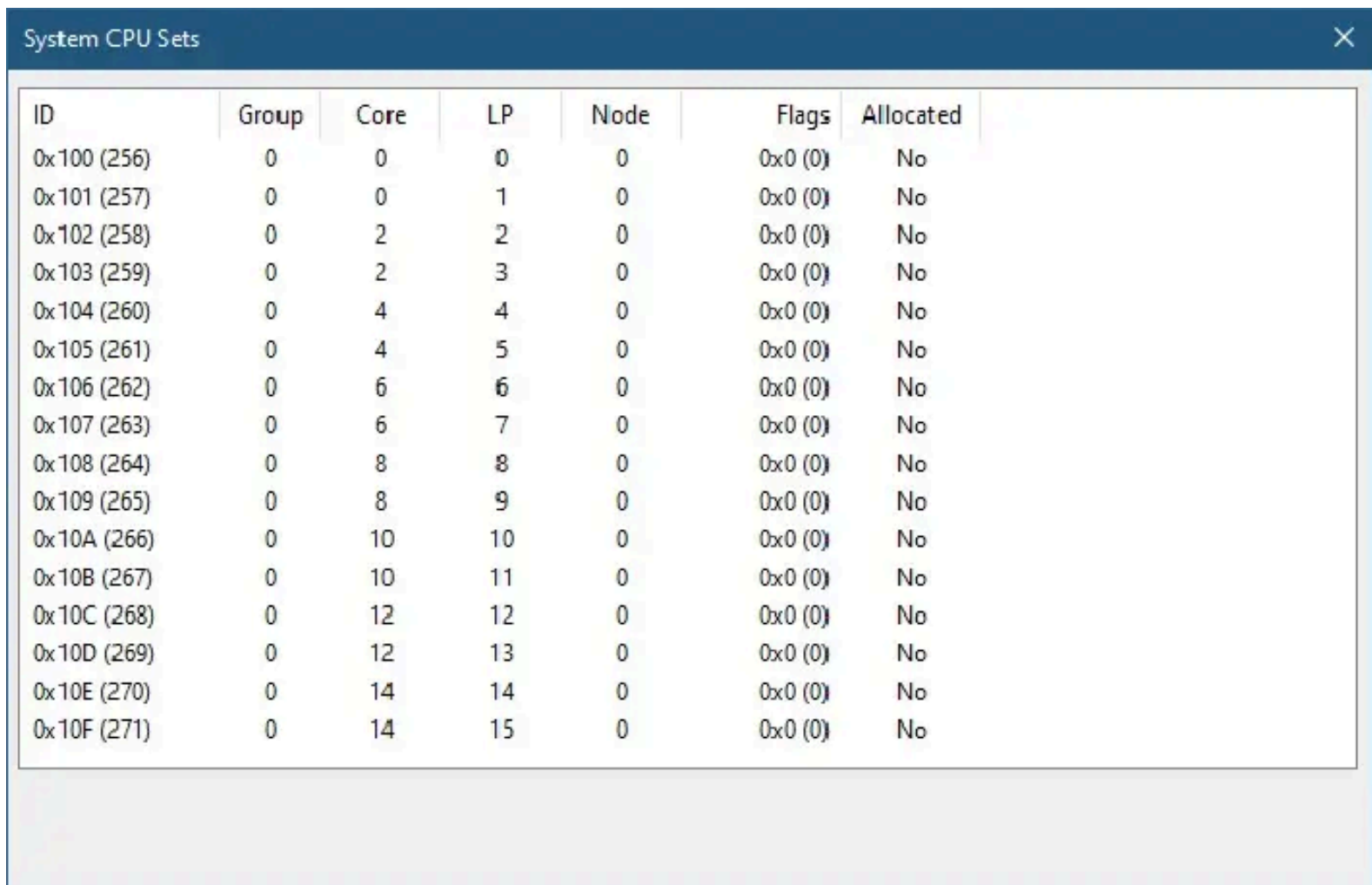
```

```

    union {
        BYTE AllFlags;
        struct {
            BYTE Parked : 1;
            BYTE Allocated : 1;
            BYTE AllocatedToTargetProcess : 1;
            BYTE RealTime : 1;
            BYTE ReservedFlags : 4;
        };
    };
    union {
        DWORD Reserved;
        BYTE SchedulingClass;
    };
    DWORD64 AllocationTag;
} CpuSet;
};
} SYSTEM_CPU_SET_INFORMATION, *PSYSTEM_CPU_SET_INFORMATION;

```

To get a sense of some of these values, run *CPUSstress* and select the *System / CPU Sets...* menu item. The system CPU set should be displayed. Figure 6-12 shows the output for a system with 1 socket, 8 cores and 16 logical processors.



ID	Group	Core	LP	Node	Flags	Allocated
0x100 (256)	0	0	0	0	0x0 (0)	No
0x101 (257)	0	0	1	0	0x0 (0)	No
0x102 (258)	0	2	2	0	0x0 (0)	No
0x103 (259)	0	2	3	0	0x0 (0)	No
0x104 (260)	0	4	4	0	0x0 (0)	No
0x105 (261)	0	4	5	0	0x0 (0)	No
0x106 (262)	0	6	6	0	0x0 (0)	No
0x107 (263)	0	6	7	0	0x0 (0)	No
0x108 (264)	0	8	8	0	0x0 (0)	No
0x109 (265)	0	8	9	0	0x0 (0)	No
0x10A (266)	0	10	10	0	0x0 (0)	No
0x10B (267)	0	10	11	0	0x0 (0)	No
0x10C (268)	0	12	12	0	0x0 (0)	No
0x10D (269)	0	12	13	0	0x0 (0)	No
0x10E (270)	0	14	14	0	0x0 (0)	No
0x10F (271)	0	14	15	0	0x0 (0)	No

Figure 6-12: System CPU Sets in *CPUSress*

Currently, *Task Manager* and *Process Explorer* don't provide information on CPU sets.

The *ID* in figure 6-12 corresponds to the `CpuSet.Id` member in `SYSTEM_CPU_SET_INFORMATION`. This is an abstract value for the ID of the CPU set itself. The first CPU set currently starts at 256 (0x100) and is incremented by one for each additional CPU set. This value of 256 was arbitrarily chosen and does not mean anything by itself. These IDs are needed, however, to change CPU sets for processes and threads, described in the following paragraphs.

The *Group* column in figure 6-12 corresponds to a process group (`CpuSet.Group` in the structure above). The *Core* column indicates the core for that CPU set (`CpuSet.CoreIndex`). Normally this is the real core number (the exact definition of this member is more elaborate and theoretical - check the documentation if interested). The *LP* column shows the logical processor number for that CPU set (`CpuSet.LogicalProcessorIndex`). The *Node* column shows the NUMA node for the CPU set (`CpuSet.NumaNodeIndex`).

For more details on other members of `SYSTEM_CPU_SET_INFORMATION`, consult the documentation.

A process can set a default CPU sets for its threads with `SetProcessDefaultCpuSets`:

```
BOOL WINAPI SetProcessDefaultCpuSets(
    _In_      HANDLE      Process,
    _In_opt_  const ULONG* CpuSetIds,
    _In_      ULONG       CpuSetIdCount);
```

The `CpuSetIds` array is expected to contain the CPU set Ids, available in the `CpuSet.Id` member of `SYSTEM_CPU_SET_INFORMATION`. A value of `NULL` removes the current CPU set assignment, meaning the CPU set constraints are removed from threads in the process that don't have specific selected CPU sets. A thread can select specific CPU sets, that may be different than its process assignment with `SetThreadSelectedCpuSets`:

```
BOOL WINAPI SetThreadSelectedCpuSets(
    _In_      HANDLE      Thread,
    _In_opt_  const ULONG* CpuSetIds,
    _In_      ULONG       CpuSetIdCount);
```

Here is an example using these functions:

```
ULONG sets[] = { 0x100, 0x101, 0x102, 0x103 };
::SetProcessDefaultCpuSets(::GetCurrentProcess(), sets, _countof(sets));
ULONG tset[] = { 0x104 };
::SetThreadSelectedCpuSets(::GetCurrentThread(), tset, _countof(tset));
```

The preceding example causes all the threads in the process to use CPU sets 0x100 to 0x103 by default, except for the current thread, which uses CPU set 0x104, essentially “escaping” its parent process CPU sets. This could be useful where a thread should have its own CPU, where other threads in the process cannot use.

CPU Sets vs. Hard Affinity

CPU sets and hard affinity may conflict with each other. In such a case, hard affinity always wins. If CPU sets contradicts hard affinity, CPU sets are ignored.

System CPU Sets

The system has its own CPU set, as can be determined with `GetSystemCpuSetInformation`, which normally returns the CPU sets available on the system. The Windows API does not provide a documented way to change it, but it is possible to do with the native `NtSetSystemInformation` call. This allows telling the “system” to avoid certain processors, so as not to interfere with user processes. This capability is used in *Game Mode*, available in Windows 10 version 1703 and later.

A detailed discussion of Game Mode is beyond the scope of this book.

Revised Scheduling Algorithm

Multiprocessor (MP) scheduling is complex: Hard affinity, ideal processor, CPU sets, power considerations, Game Mode, and other aspects all make scheduling decisions on MP complicated, to say the least. The ready queue (actually an array of 32 queues, one per priority) described in the section “Scheduling Basics” is extended on MP systems: each processor has its own ready queue. Additionally, on Windows 8 and later, there are shared ready queues for groups of processors (currently a maximum of 4 per group). This allows the scheduler to have more options when it needs to locate a processor for a ready thread that is attached to a shared ready queue (per-CPU ready queues are still used for threads that have hard affinity constraints).

the above details can be changed by Microsoft in future versions of Windows. The text is there to give some ideas on the complexity of scheduling.

A revised, simplified, MP scheduling algorithm is presented in figure 6-13. It assumes no affinity or CPU set constraints, no power or other special considerations.

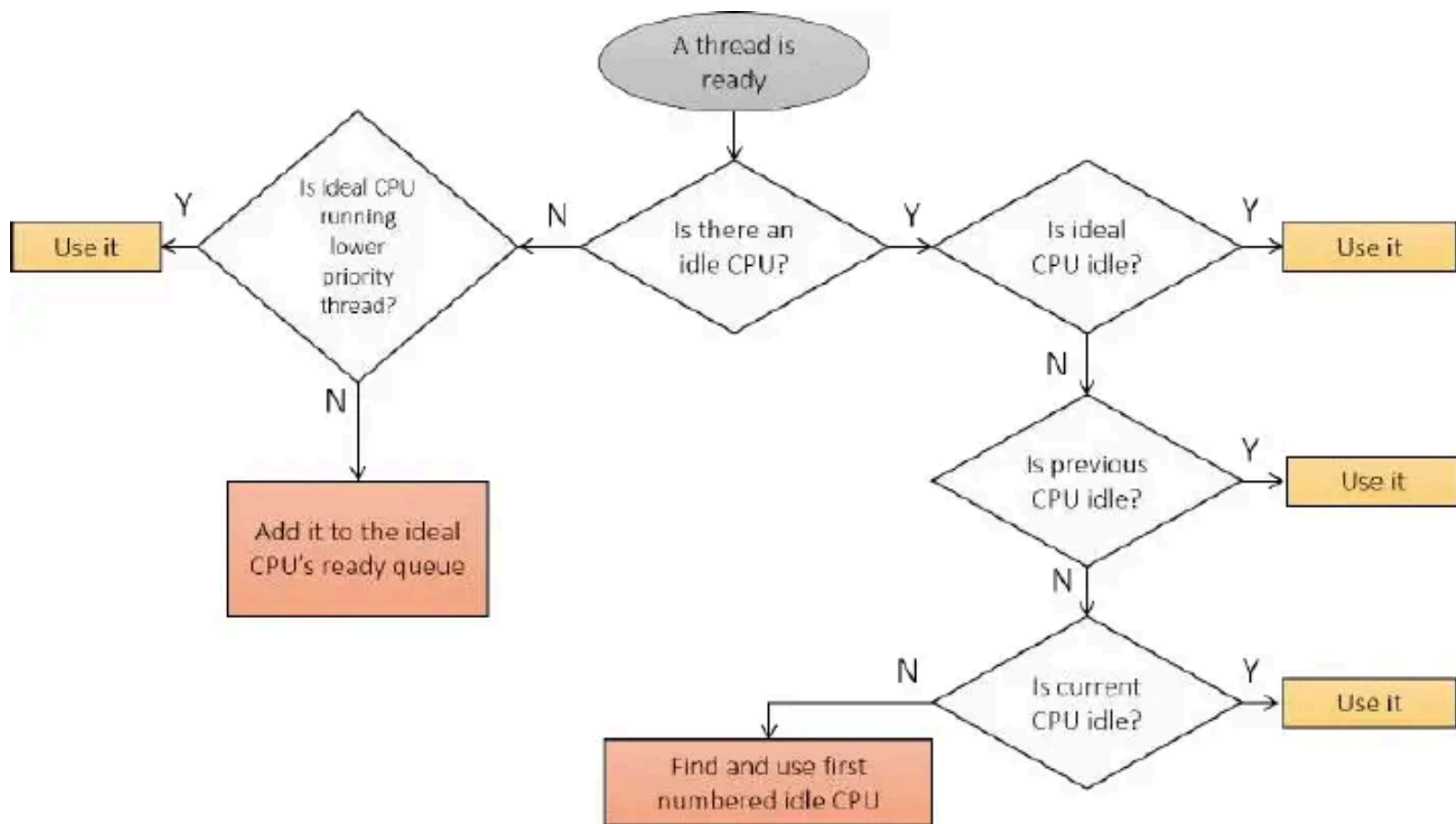


Figure 6-13: MP simplified scheduling

As can be seen in figure 6-13, the ideal processor is the preferred processor to use, followed by the last processor it ran on (processor's cache may still contain data used by that thread). If all processors are busy, the scheduler does not preempt the first processor that runs a low-priority thread; that would be inefficient, as many processors may be needed to be searched. Instead, the thread is put in the (shared) ready queue of its ideal processor.



The next Windows version, 2004 (April 2020) may change the order of check between ideal processor and previous processor. Regardless, from a developer's perspective, it matters very little.

Observing Scheduling

Scheduling changes are fairly frequent, but they can be observed by using tools. In the following sections I'll describe some experiments you can use to investigate and experience scheduling. This section is optional and can be safely skipped.

General Scheduling

To get a general sense of scheduling we, can use *Performance Monitor*. Run *CPUSstress* and terminate two threads so that only two remains. Activate both threads (figure 6-14).

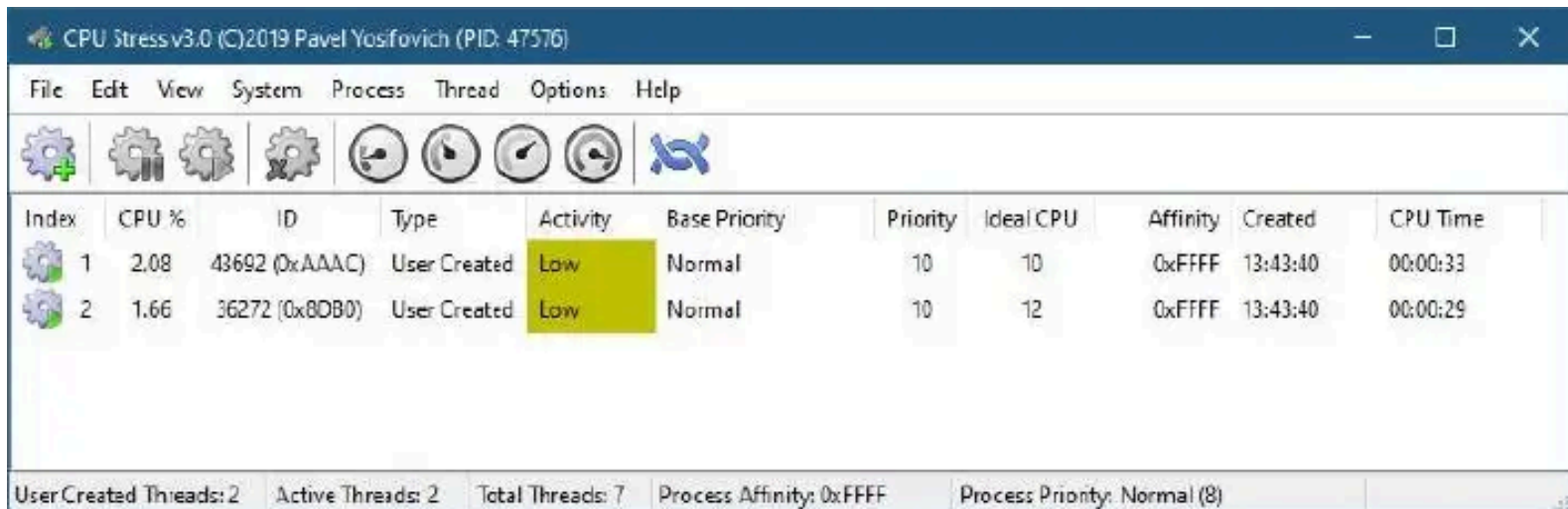


Figure 6-14: *CPUSstress* with two active threads

Now open *Performance Monitor* (type *perfmon* at Run prompt or just search for it). The *Performance* console appears. Click on the *Performance Monitor* item (figure 6-15).

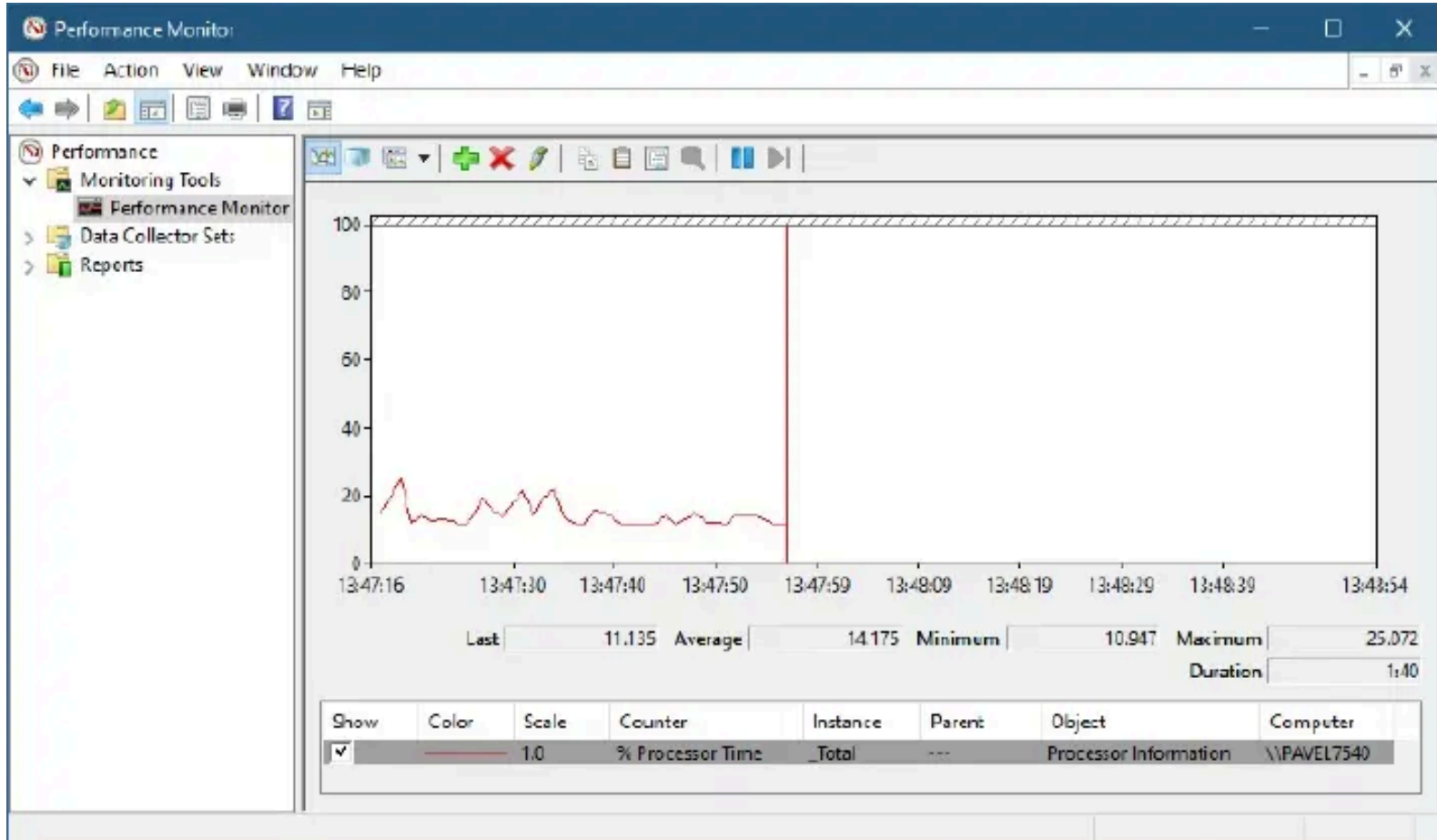


Figure 6-15: *Performance Monitor*

Performance Monitor is a built-in tool that can show *performance counters*, which are just numbers, exposed by various system components. technically, any application can register and expose performance counters. In the following examples we use some of these built-in counters that are related to scheduling.

Delete the default counter, and Click the *Add* (green plus button) to add new counters. Search for the *Thread* category and open it (figure 6-16).

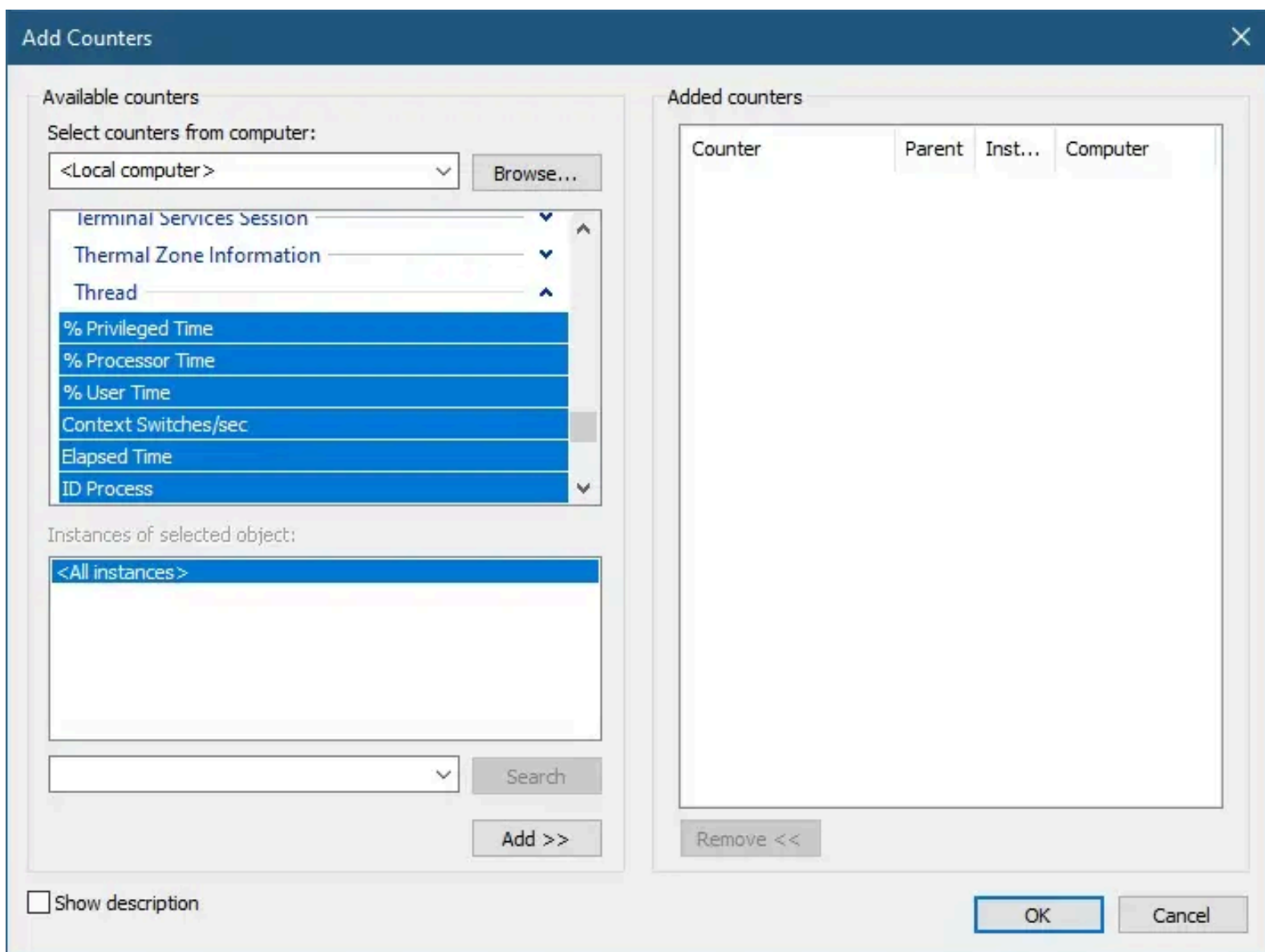


Figure 6-16: The *Thread* performance counter category

Now select the following counters: *Priority Current* and *Thread State* (use the Control key to multi-select) (figure 6-17).

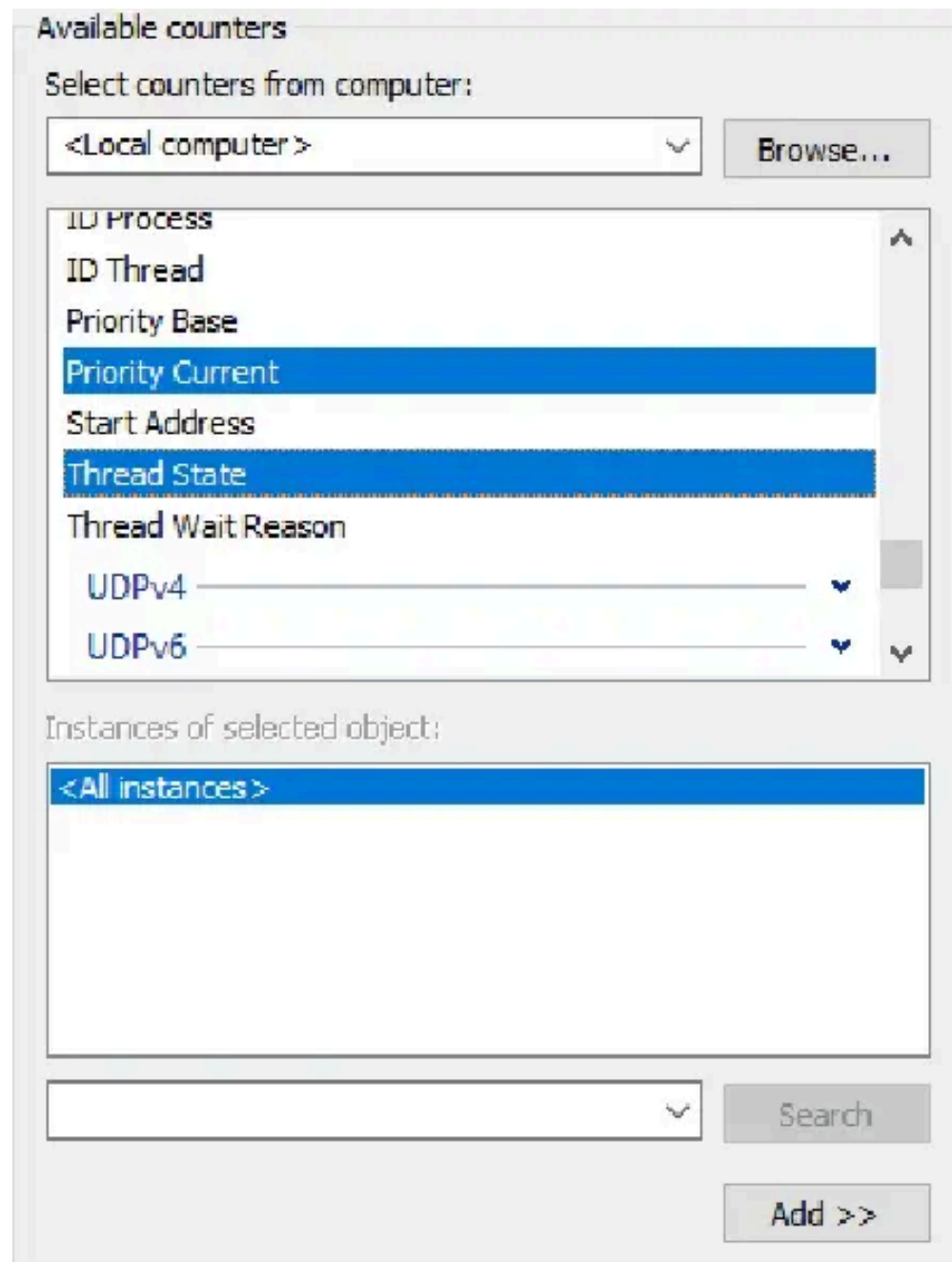
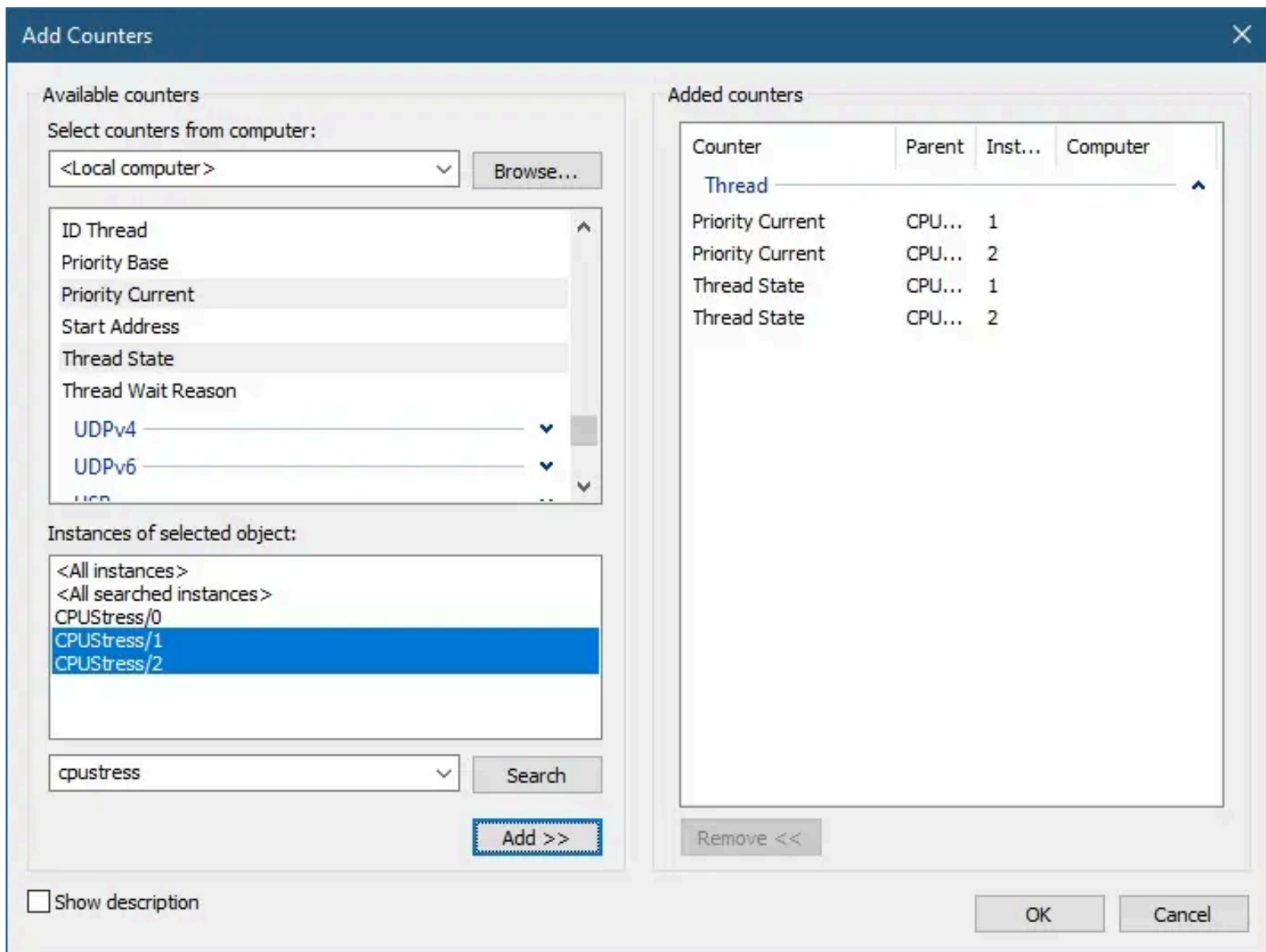


Figure 6-17: Counters selected from the *Thread* category

In the lower search box, type *CPustress* and press ENTER. A list of threads should show up. Select the thread 1 and 2 (these should be the numbers shown to left in *CPUStress*). Click *Add* (figure 6-18) and then *OK*.

Figure 6-18: Two threads selected in *Threads* category

4 graphs are now shown, with the current priority and states of the two worker threads from *CPUSress*. Right click an empty graph region and select *Properties*. Switch to the *Graph* tab and change the vertical scale to be between 0 and 16 (figure 6-19). Click *OK*.

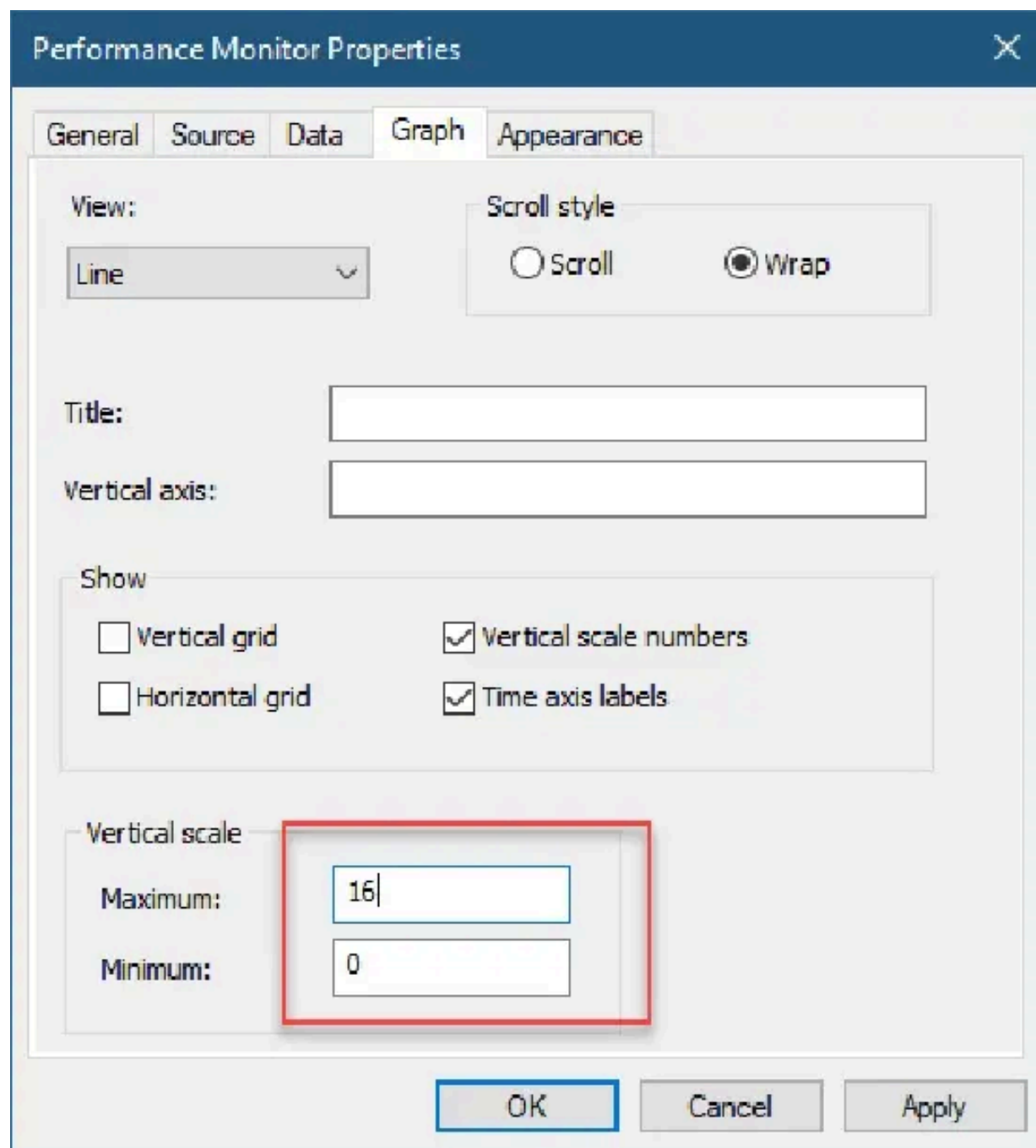


Figure 6-19: Changing graph scale

Now the priorities and states should be more easily identifiable (figure 6-20).

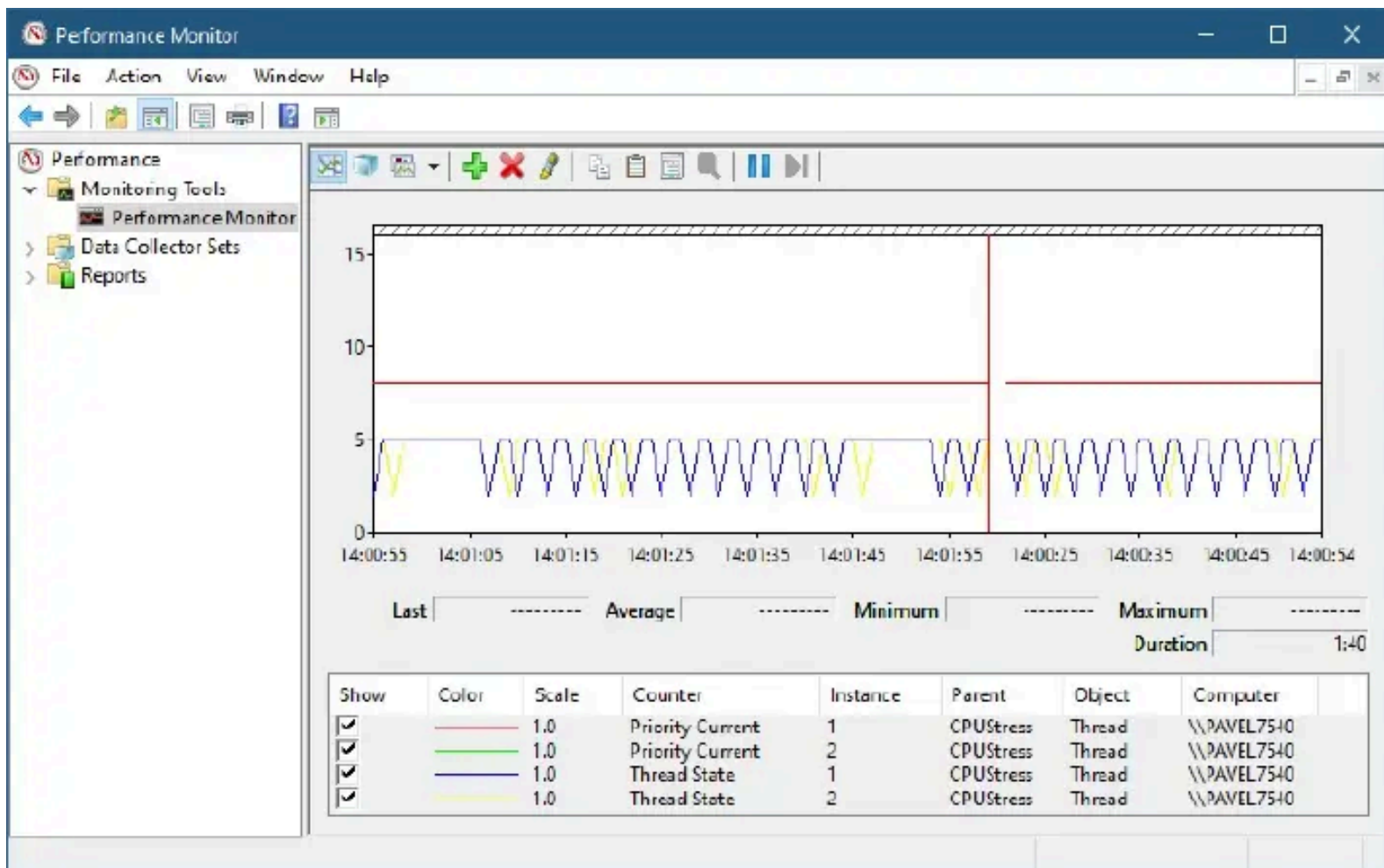


Figure 6-20: The counters at work

Notice the priority of the threads is 8 (red and green lines in figure 6-20, green is obscured by the red). The thread states alternates between 2 and 5. Here is the main state numbers: Running=2, Ready=1, Waiting=5. Now switch to the *CPUSress* application. Notice the thread priorities jump to 10. If you switch to another application, it drops back down. You can change the activity level of one thread to *high* and see the effect. (figure 6-21). You can play with priorities as well.

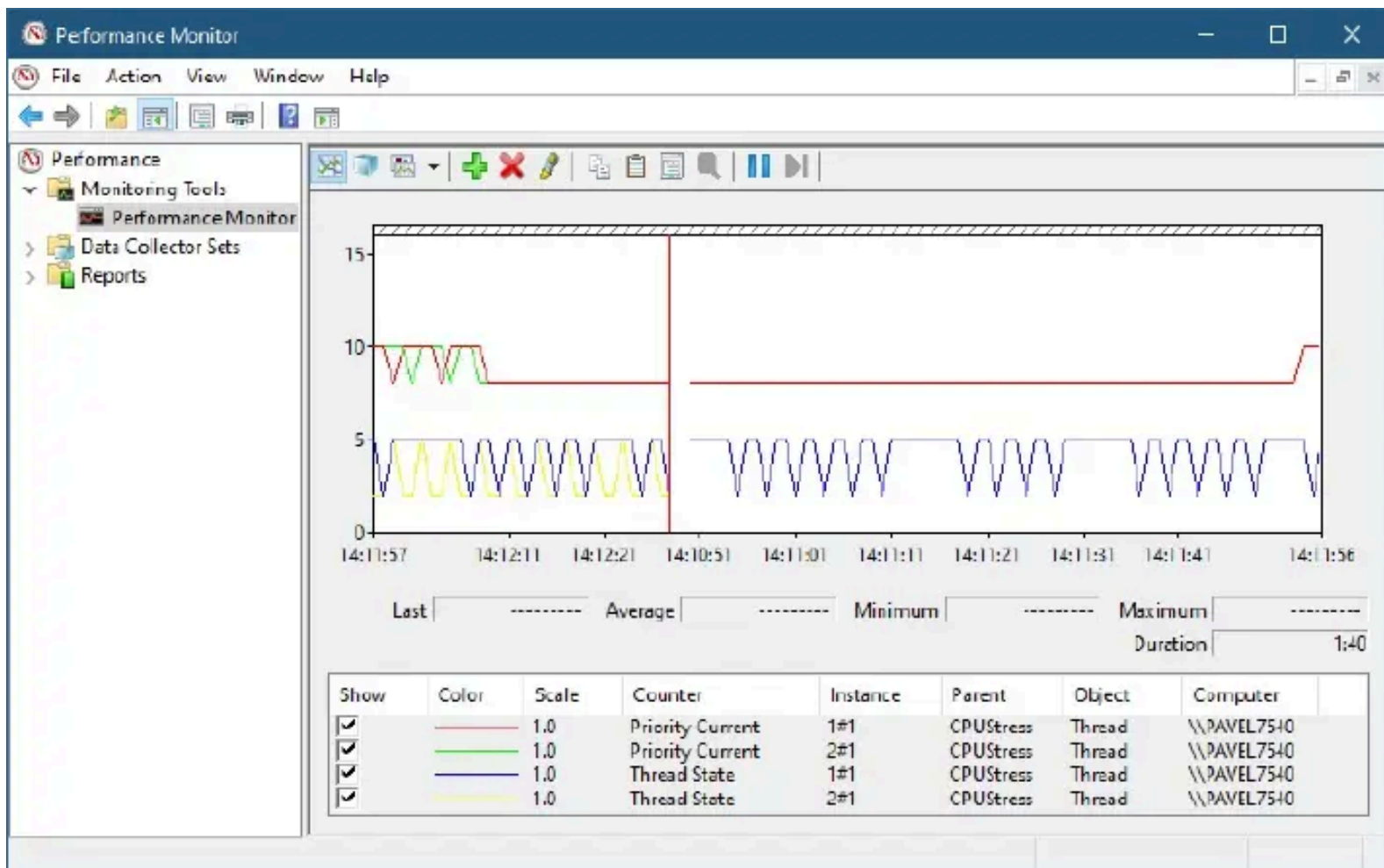
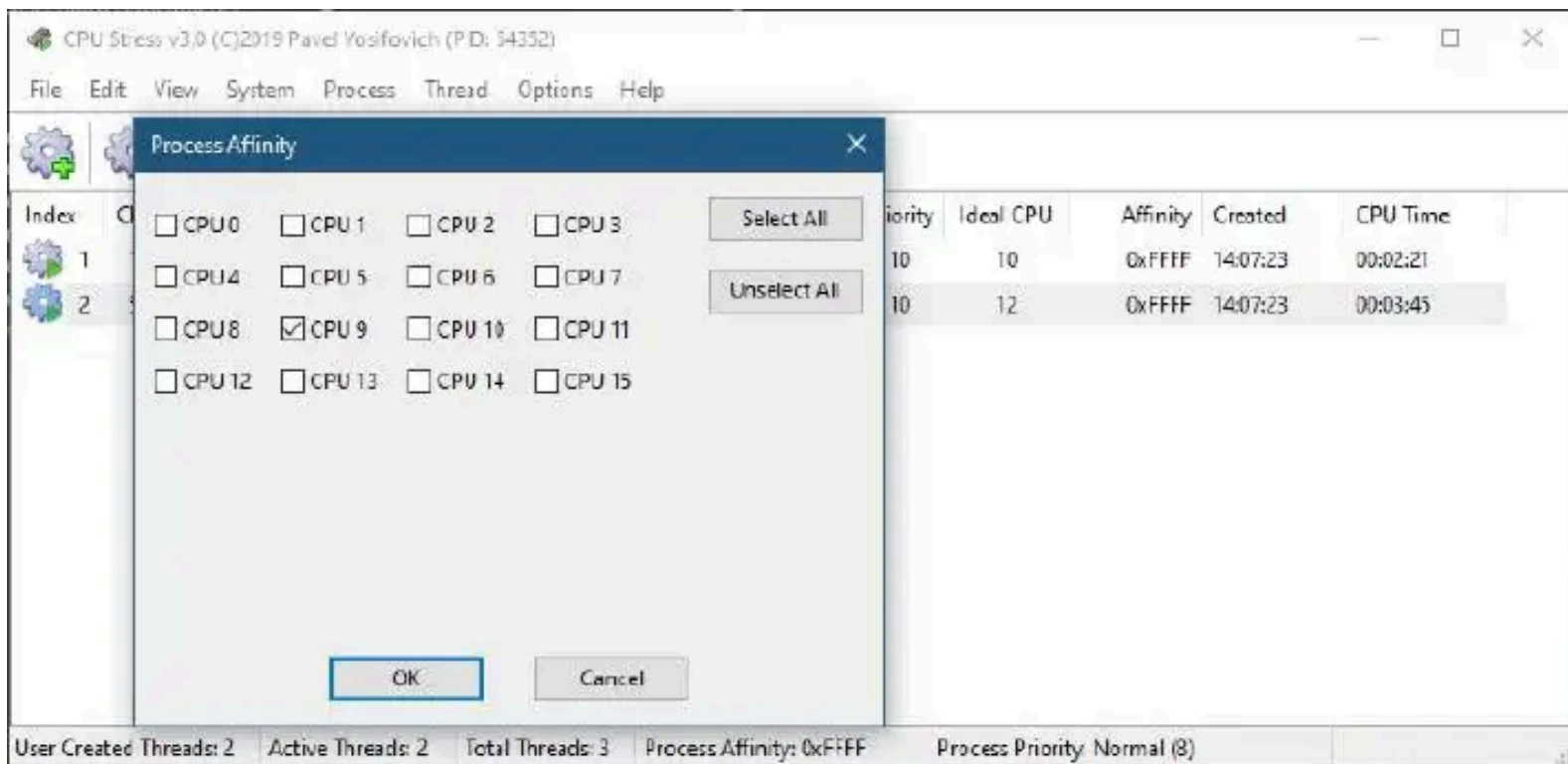


Figure 6-21: state changes in threads

Hard Affinity

You can test hard affinity by continuing from the previous experiment. *CPUSress* allows restricting affinity (you can use *Task Manager* as well). Select *Process/Affinity* from the menu and choose a single CPU as hard affinity (doesn't matter which) (figure 6-22).

Figure 6-22: Process affinity in *CPUSress*

You should see threads going into the *ready* state from time to time, as both threads fight for the single CPU. Increase their activity to *Maximum* and observe the threads alternate between states 2 (running) and 1 (ready) (figure 6-23).

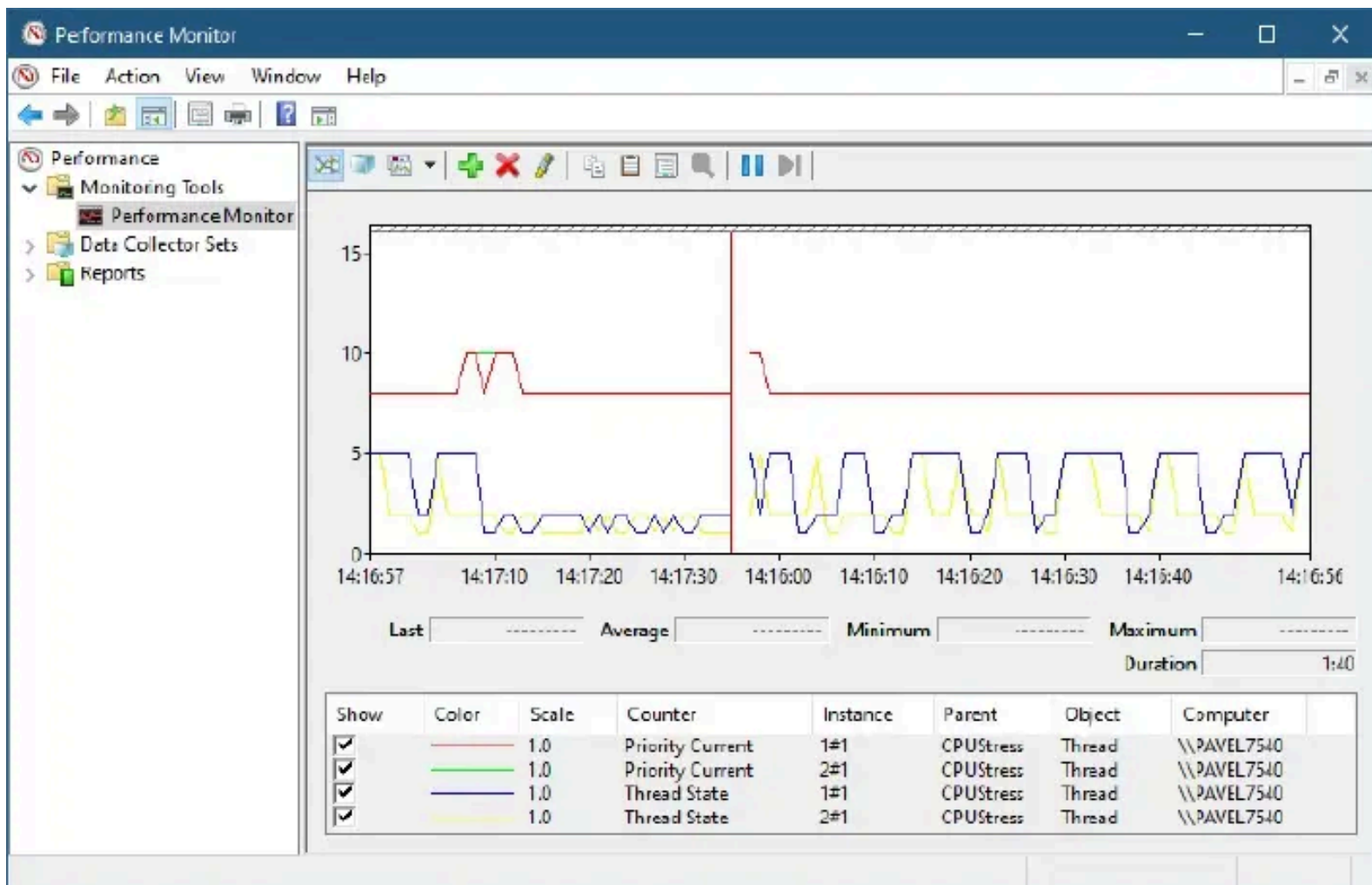


Figure 6-23: Maximum activity with a single processor affinity



Performance Monitor only updates every 1 second, which means several quanta elapse in that time. This means what you see is not completely accurate, but it gives the general idea.

CPU Sets

Observing CPU sets requires a different tool, one that can show CPU numbers used by threads. We'll use the *Windows Performance Recorder* (WPR) from the Windows SDK's *Windows Performance Toolkit*.

Search for *Windows Performance Recorder* (*wprui.exe*). If you can't find it, most likely it isn't installed. Run the Windows 10 SDK installation again and add *Windows Performance Toolkit*.

In the main WPR UI, select *First Level Triage* only. This captures CPU, memory, I/O and other events, from which we'll look into CPU related events (figure 6-24).



WPR uses *Event Tracing for Windows* (ETW) to capture various events emitted by various system components including the scheduler.

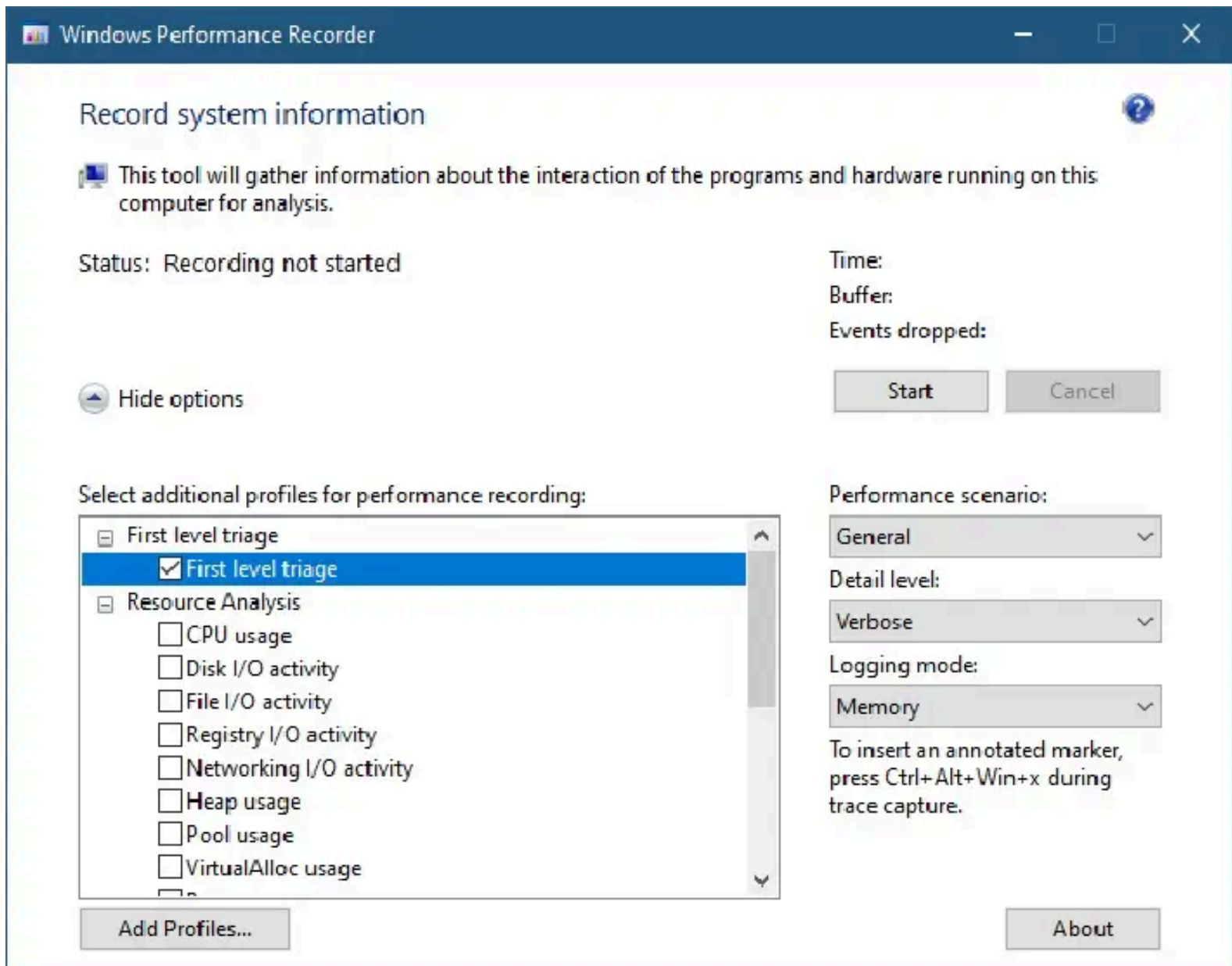


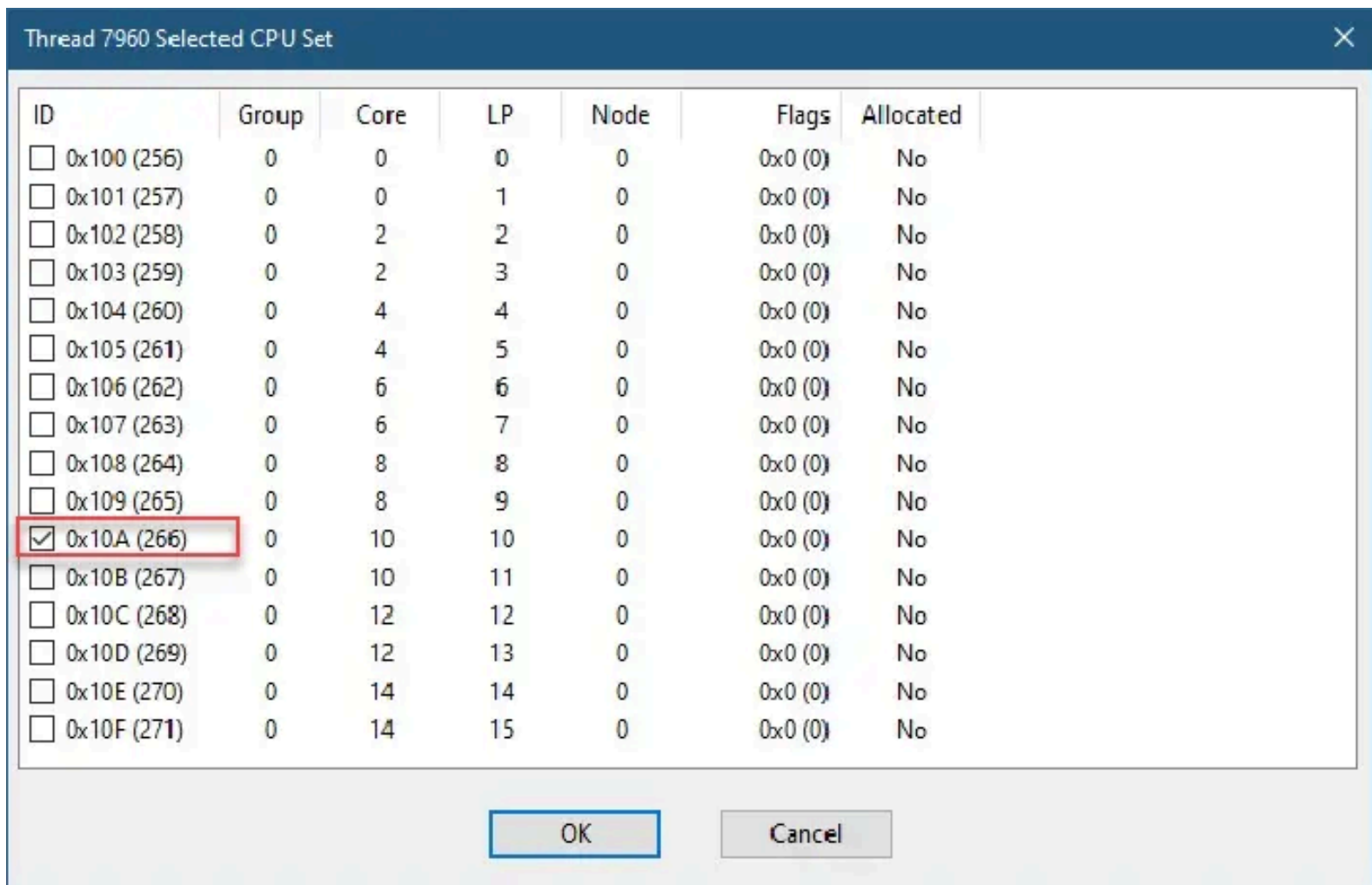
Figure 6-24: WPR main user interface

Go to *CPUSstress* and reset the affinity to all processors. Also, set the process CPU sets to the first 4 processors using the *Process/CPU Sets* menu item (figure 6-25).

Process CPU Set						
ID	Group	Core	LP	Node	Flags	Allocated
☒ 0x100 (256)	0	0	0	0	0x0 (0)	No
☒ 0x101 (257)	0	0	1	0	0x0 (0)	No
☒ 0x102 (258)	0	2	2	0	0x0 (0)	No
☒ 0x103 (259)	0	2	3	0	0x0 (0)	No
☐ 0x104 (260)	0	4	4	0	0x0 (0)	No
☐ 0x105 (261)	0	4	5	0	0x0 (0)	No
☐ 0x106 (262)	0	6	6	0	0x0 (0)	No
☐ 0x107 (263)	0	6	7	0	0x0 (0)	No
☐ 0x108 (264)	0	8	8	0	0x0 (0)	No
☐ 0x109 (265)	0	8	9	0	0x0 (0)	No
☐ 0x10A (266)	0	10	10	0	0x0 (0)	No

Figure 6-25: Restricted CPU sets in *CPUSstress*

Next, use the *Thread/Selected CPU Sets...* menu item to set one of the worker threads to use some other CPU (figure 6-26), where CPU 10 is selected.

Figure 6-26: Selected CPU sets for a thread in *CPUSTress*

Make sure the two threads in *CPUSTress* are active. Now go back to WPR and click the *Start* button. Wait 2 or 3 seconds and click *Stop*. Wait for the processing to finish, and once it does, open the trace in *Windows Performance Analyzer* (WPA) with the *Open in WPA* button.



Windows Performance Analyzer (WPA) is an analysis tools for ETW captures. It's fairly complex and versatile, and the following information barely scratches the surface of this powerful tool. WPA is beyond the scope of this book.

When the trace opens, navigate in the left pane to *Computation / CPU Usage (Precise) / CPU Utilization by Process, Thread, Activity*. You should see something like figure 6-27.

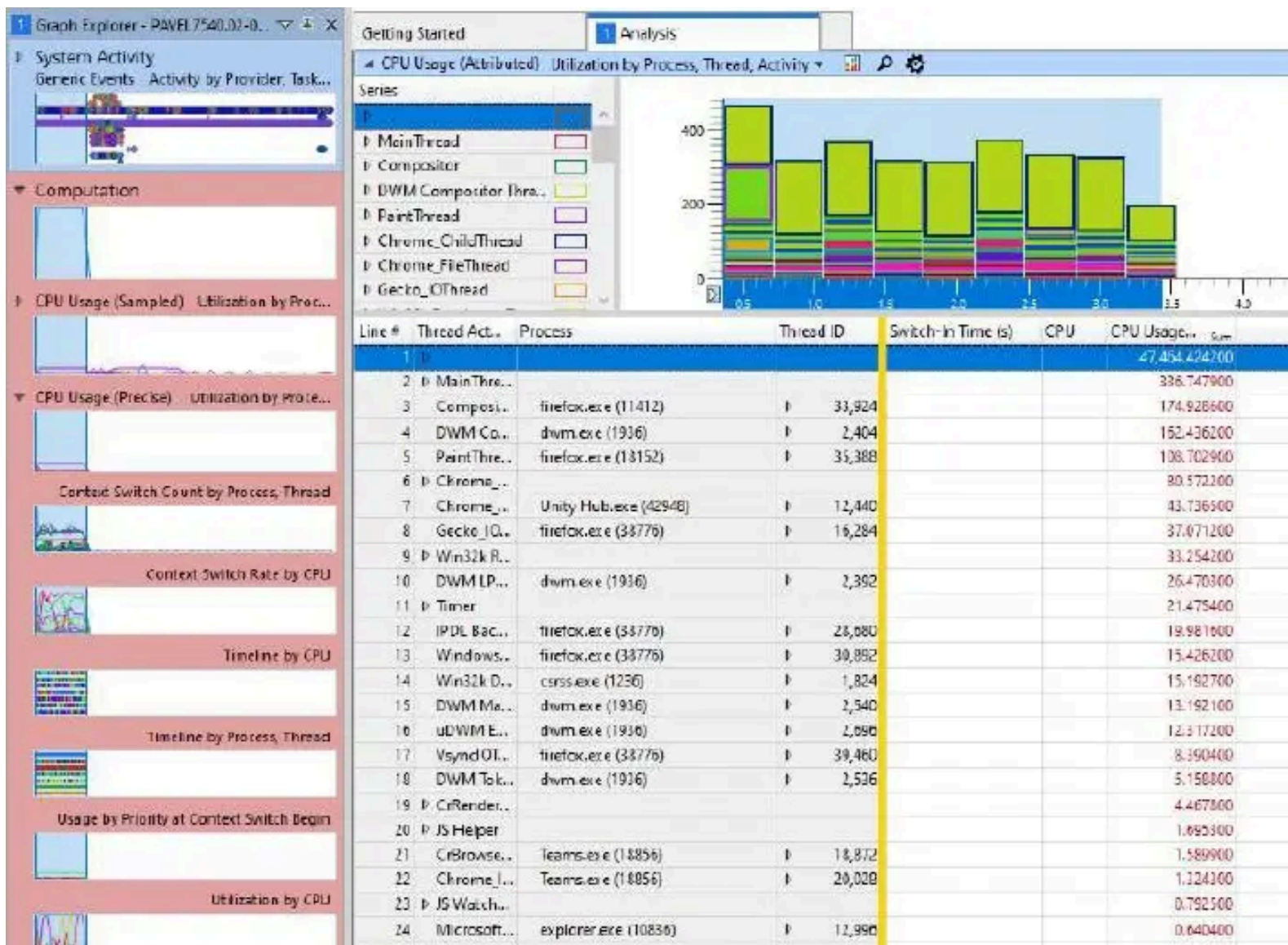
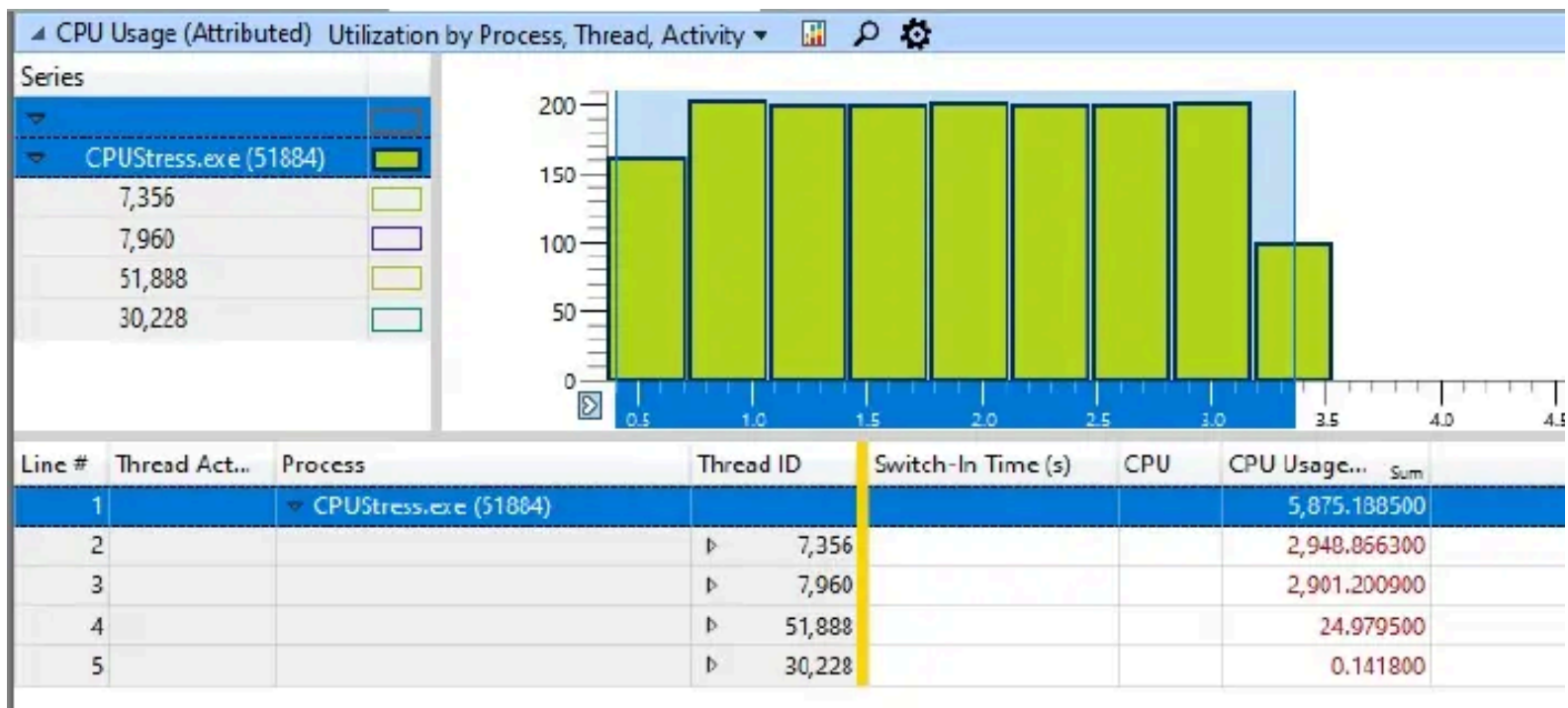


Figure 6-27: Typical UI for WPA

ETW traces are always system-wide, so we first need to filter to our process of interest - *CPUSTress*. Expand the unnamed Series tree node in the top left and locate *CPUSTress*. Right-click it and select *Filter to Selection*. The view should clear out, leaving *CPUSTress* information only. Expand the process node, revealing thread nodes (figure 6-28).

Figure 6-28: WPA filtered to *CPUStress* process

If you expand a thread which did not have its selected CPU sets altered, you should see CPU numbers 0 to 3 (the first four processors set for the process CPU set) (figure 6-29). On the other hand, expanding the thread with its own selected CPU set should only show that CPU as being utilized (figure 6-30).

Process	Thread ID	Switch-In Time (s)	CPU	CPU Usage...	Sum
▼ CPUStress.exe (51884)				5,875.188500	
	▼ 7,356			2,948.866300	
		0.834771600	1	317.949800	
		2.413675300	3	282.046600	
		2.803822300	2	216.938600	
		3.154981700	2	197.705800	
		1.154337900	1	120.316100	
		1.707817700	3	114.882800	
		0.617758100	3	108.939400	
		2.038749800	3	108.928400	
		1.490872600	3	108.849500	
		0.509712600	2	108.026900	
		1.382759900	0	108.024200	
		1.274703500	1	107.989900	
		1.599764000	1	107.983500	
		3.020830700	2	107.965200	
		2.255706000	1	107.950600	
		1.822775200	2	107.950400	
		1.930734900	3	107.940600	
		2.695831000	3	107.920200	
		2.155405300	3	100.256400	
		0.779550300	0	55.144700	

Figure 6-29: Thread with no assigned CPU sets

Line #	Thread Act...	Process	Thread ID	Switch-In Time (s)	CPU	CPU Usage...	Sum
1		▼ CPUStress.exe (51884)				5,875.188500	
2			7,356			2,948.866300	
3			7,960			2,901.200900	
4				1.490907400	10	331.789300	
5				0.456306800	10	270.421400	
6				1.154360600	10	228.355800	
7				3.128848900	10	223.825400	
8				2.695765500	10	215.960100	
9				1.822818900	10	215.864300	
10				0.942827600	10	211.402400	
11				2.161720300	10	201.937400	
12				2.413667700	10	174.083400	
13				2.038749600	10	108.931300	
14				1.382794000	10	108.073300	
15				0.834804000	10	108.022900	
16				3.020802400	10	107.995200	
17				0.726769300	10	107.954100	
18				2.588921600	10	106.815700	
19				2.911758800	10	61.990800	
20				2.363697800	10	49.929800	
21				2.973776300	10	46.984500	
22				2.147724600	10	13.920800	
23				0.453055800	10	2.118000	

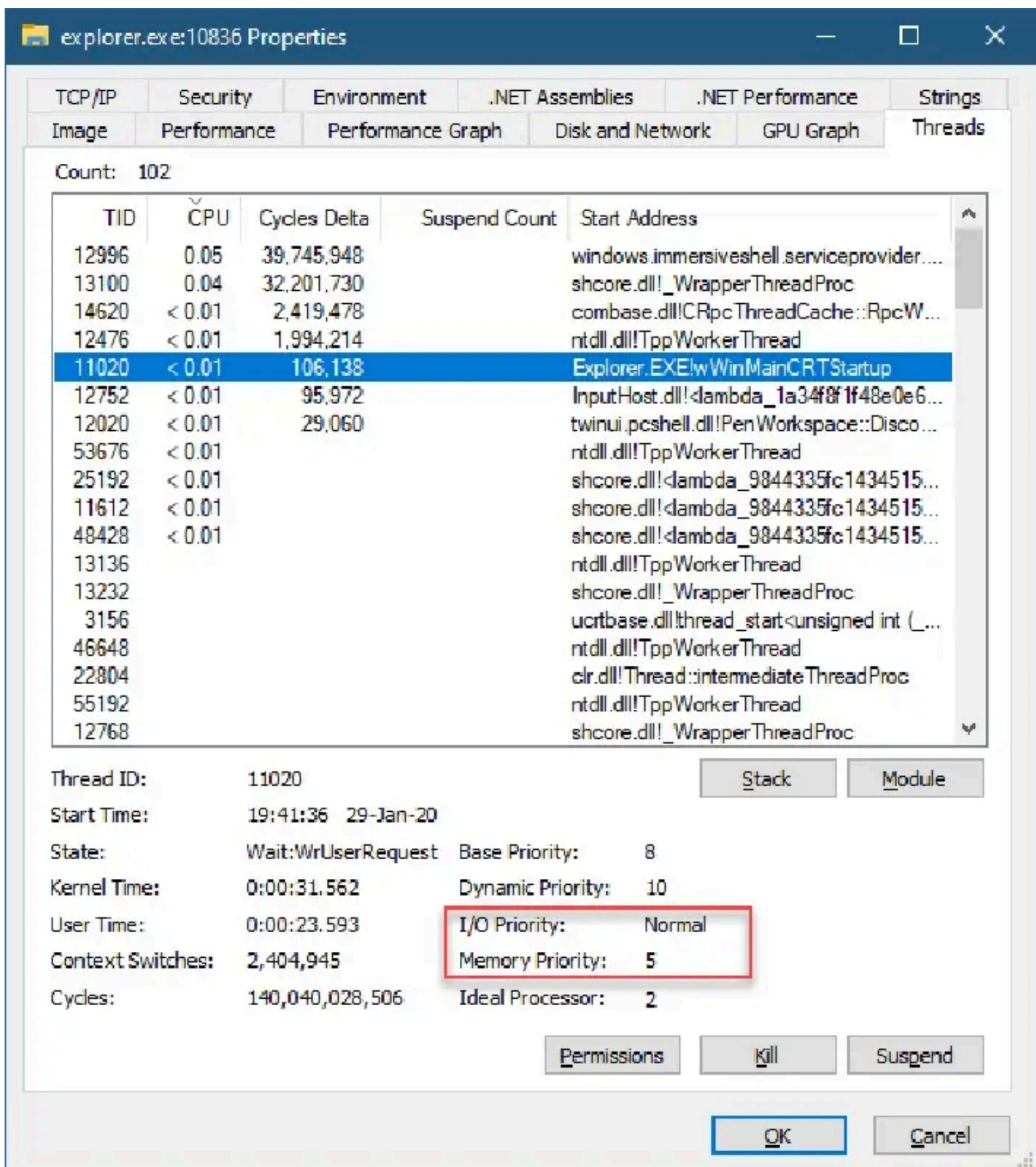
Figure 6-30: Thread with assigned CPU set to CPU 10

Background Mode

Some processes are naturally more important than others. For instance, if a user works with Microsoft Word, she probably expects her interaction and usage of Word to be very good. On the other hand, processes such as backup application, anti-virus scanners, search indexers and similar are not as important and should not interfere with the user's main applications.

One way for these background applications to limit their impact is to lower their CPU priority. This works, but CPU is just one type of resource used by processes. Other resources include memory and I/O. This means reducing the CPU priority of threads or the priority class of a process may not be enough to reduce the impact of such processes.

Windows offers the concept of *Background Mode*, where a thread's CPU priority drops to 4 and the *memory priority* and *I/O priority* drop as well. For example, looking at Windows Explorer in *Process Explorer* in the *Threads* view show memory and I/O priority as well as CPU priority (figure 6-31). I/O priority has a default value of "Normal" and memory priority's default value is 5 (possible values are 0 to 7).

Figure 6-31: Memory and I/O priority in *Process Explorer*

The exact definition of memory priority and I/O priority are not important for this discussion. We'll discuss memory priority in a later chapter. For I/O priority - intuitively - higher levels get precedence over lower levels when accessing I/O.

As a contradictory example, examine figure 6-32, showing threads for a *SerachFilterHost.exe* process. Notice its memory and I/O priority.

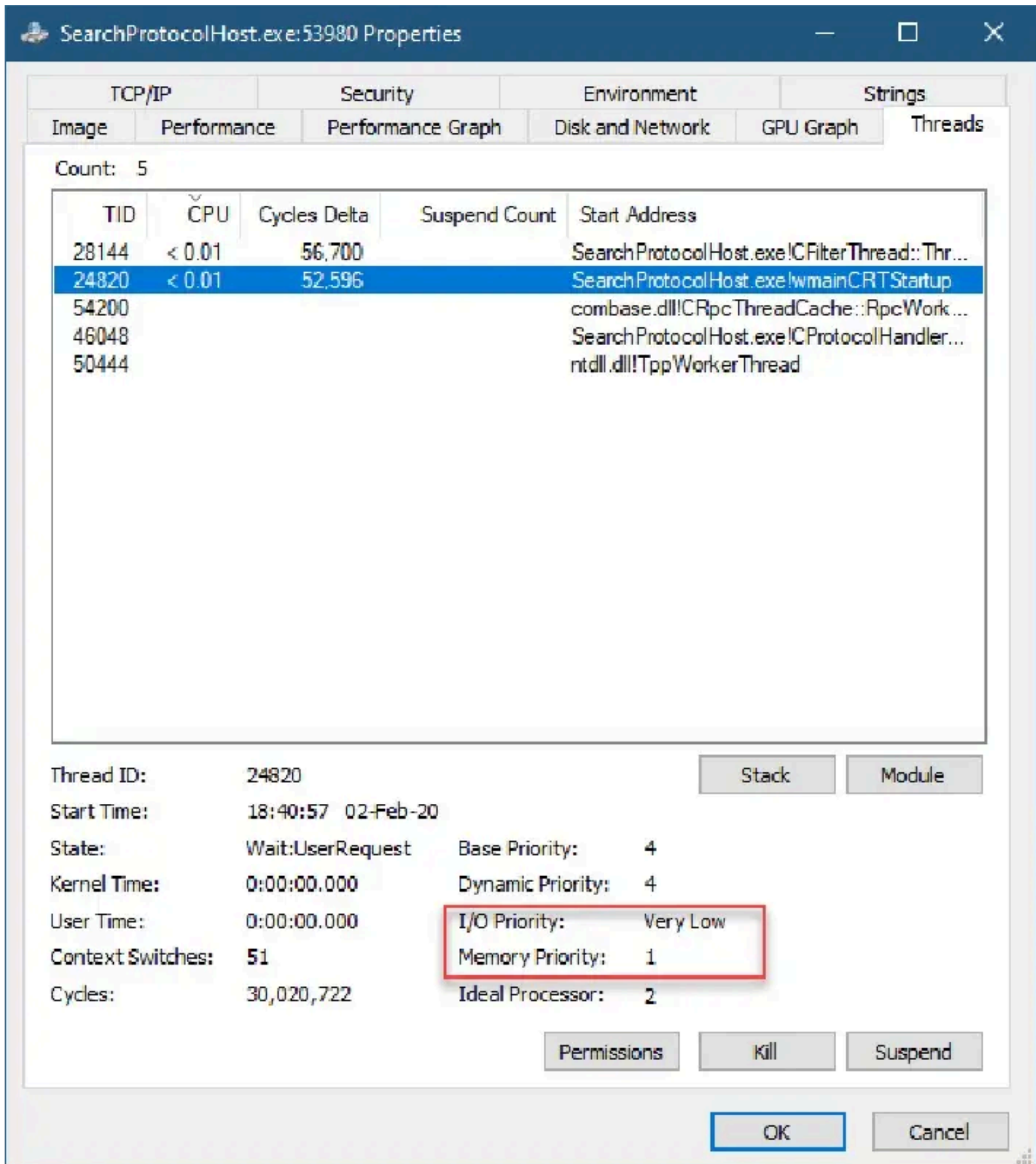


Figure 6-32: Low memory and I/O priority in *Process Explorer*

This *SearchProtocolHost.exe* process lowered its I/O and memory priority as well as its CPU priority in one swoop using a call to `SetPriorityClass` with the special `PROCESS_MODE_BACKGROUND_BEGIN` value like so:

```
::SetPriorityClass(::GetCurrentProcess(), PROCESS_MODE_BACKGROUND_BEGIN);
```

The handle to the process must point to the current process, otherwise the call fails. The background mode begins for all threads in the process until the complementary call is made with `PROCESS_MODE_BACKGROUND_END`:

```
::SetPriorityClass(::GetCurrentProcess(), PROCESS_MODE_BACKGROUND_END);
```

Similarly, the call can be made on a thread basis with the standard `SetThreadPriority` with the special values `THREAD_MODE_BACKGROUND_BEGIN` and `THREAD_MODE_BACKGROUND_END`. Here too, the thread handle must reference the current thread for the call to succeed.

The fact that the above calls require the current process/thread mean that a thread or process cannot be “forced” into background mode; rather, the thread or process itself should be a “good citizen” and enter background mode willingly.

Process Explorer does allow forcing a process into background mode. Right-click a process and select *Set Priority/Background*.

Priority Boosts

As we’ve seen in the section “Scheduling Basics”, the priority is the determining factor where scheduling is concerned. However, Windows employs several tweaks to priority called *priority boosts*. These temporary increase in priority is designed to make scheduling a little more “fair” in some sense, or to provide a better experience for the user. In this section, I’ll discuss some of the common priority boost reasons. In any case, don’t depend on these boosts, as they may be removed and new ones may appear in future versions of Windows.



Remember that threads in the real-time range (priorities 16 to 31) never have their priority boosted.

Completing I/O Operations

When a thread issues a synchronous I/O operation, it enters a wait state until the operation completes. Once it completes, the device driver responsible for the I/O operation has an opportunity to boost the requesting thread's priority to increase its chances of running sooner, now that the operation finally completes. The priority boost (if applied) increases the thread's priority by the amount at the driver's discretion, and the priority decays by one level every quantum that the thread manages to run, until the priority drops back to its base level. Figure 6-33 shows a conceptual view of this process.

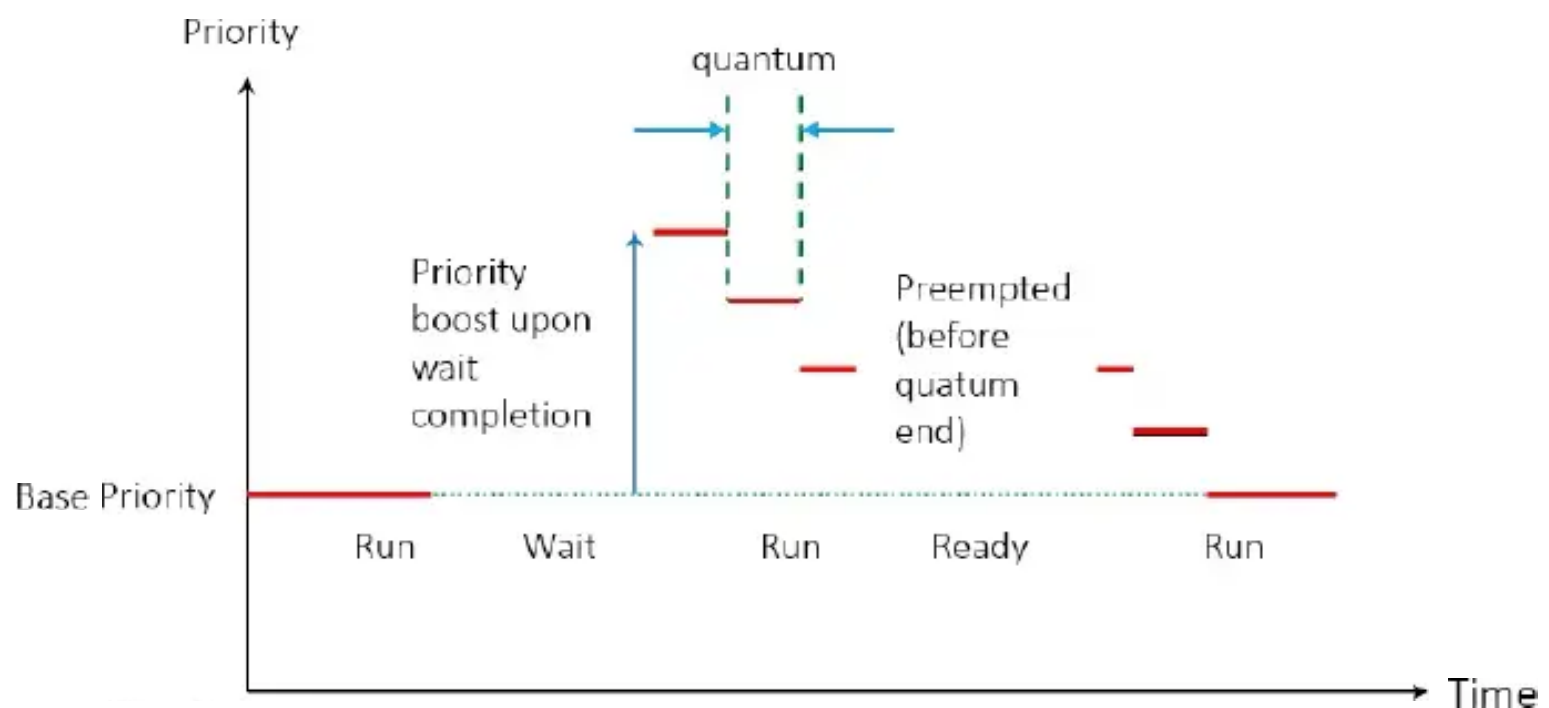


Figure 6-33: Thread priority boost and decay

Foreground Process

There is always an active window on a system - the one usually with a caption in a different color. This window was created by a thread, which is part of a process. This process is sometimes referred to as the *Foreground Process*. In a foreground process on a system configured with short quanta (the default for client versions of Windows): when a thread completes a wait on a kernel object, it gets a +2 boost to its priority. This priority decays after a single quantum to its base level.

GUI Thread Wakeup

When a thread that has a user interface receives a windows message (typically its call to `GetMessage` returns), it gets a +2 boost to improve its chances of running sooner. This priority decays after a single quantum to its base level.

Starvation Avoidance

Threads that are in the *ready* state for at least 4 seconds, get a large boost to priority 15 for a single quantum of execution, after which the priority drops back to its original level. This allows threads in low priority to make some forward progress even on a relatively busy system.

Other Aspects of Scheduling

This section looks at other aspects of scheduling not yet discussed.

Suspend and Resume

A thread can be created in a suspended state by specifying the `CREATE_SUSPENDED` flag as part of `CreateThread`. This allows the caller to prepare the thread for execution, such as manipulating its priority or affinity before actually executing useful code. Using this flag creates the thread with a suspend count of 1. More generally, a thread can be suspended by calling `SuspendThread`:

```
DWORD WINAPI SuspendThread(_In_ HANDLE hThread);
```

The thread handle must have the `THREAD_SUSPEND_RESUME` access mask for the call to succeed. The function increments the suspend count of the thread, and suspends its execution. The function returns the previous suspend count of the thread or `(DWORD)-1` if it fails. There is a maximum suspend count a thread can have defined as `MAXIMUM_SUSPEND_COUNT` (defined as 127). A thread can suspend itself, but it cannot resume itself.

Once suspended, the thread can be resumed with `ResumeThread`:

```
DWORD WINAPI ResumeThread(_In_ HANDLE hThread);
```

ResumeThread decrements the thread's suspend count, and if it's zero, it becomes eligible for execution. This function is required if a thread has been created with `CREATE_SUSPENDED`.

Generally, suspending a thread is a bad idea, since there is no way to know where exactly the suspension occurs. For example, the thread might have acquired a lock that other threads are waiting on. Suspending such a thread before it releases the lock causes a deadlock, as other threads contending for the lock will wait indefinitely.



Thread synchronizations and locks are discussed in the next chapter.

Suspending and Resuming a Process

A process is not scheduled, threads do. However, it may be desirable at times to suspend all threads in a process at once. This functionality is used for example in the case of UWP processes, which get all their threads suspended when the application goes to the background. The Windows API does not provide a function for this. It's technically possible to iterate over all threads in a process and call `SuspendThread`, but it's risky at best. A new thread might start while iterating, which would likely miss that thread.



UWP process suspension builds on an undocumented job object feature called *Deep Freeze*.

The native API however, does provide the (undocumented) `NtSuspendProcess` function defined like so:

```
NTSTATUS NtSuspendProcess(_In_ HANDLE hProcess);
```

Although undocumented, it's been around forever, so it's fairly safe to use. If you do use it, don't forget to add `extern "C"` to the function definition to let the linker know it's a C function. Also, add the *NtDll* import library to the project's linker input libraries or in source code like so:

```
#pragma comment(lib, "ntdll")
```

The inverse function exists as well, aptly named `NtResumeProcess`:

```
NTSTATUS NtResumeProcess(_In_ HANDLE hProcess);
```

Process Explorer offers a right-click operation on a process to suspend/resume it. It uses the above functions internally.

Sleeping and Yielding

A thread can voluntarily relinquish the remainder of its quantum by entering sleep:

```
void Sleep(_In_ DWORD dwMilliseconds);
```

The thread goes into a wait state for **approximately** the number of milliseconds requested. A value of zero is valid, and causes the scheduler to execute the next thread in the queue which has the same priority. If there is none, the thread continues execution. Another legal value is `INFINITE` which causes the thread to sleep forever; this is hardly ever useful.



The accuracy of the sleep interval actually depends on whether the internal timer resolution has been altered. Normally it should be the same as a clock tick used for scheduling, but usually it's much less because some other thread requested it. The output of the *clockres.exe* *Sysinternals* utility shows the current timer interval, that affects (among other things) sleep time accuracy.

As an alternative to calling `Sleep(0)`, a thread can call `SwitchToThread`:

```
BOOL SwitchToThread(void);
```

`SwitchToThread` tells the scheduler to schedule the next ready thread, even if its priority is lower than the current thread. The function returns `TRUE` if the scheduler is able to comply; otherwise, the thread continues execution and the function returns `FALSE`.

Summary

In this chapter we looked at various aspects of scheduling. From priorities and how they can be set, to simple single CPU scheduling, up to multiprocessor considerations, with affinities and CPU sets. In the next chapter, we'll look at thread synchronization, where threads must coordinate their efforts and the various ways they can do so.